Temporally Extended Mixture-of-Experts Models

Zeyu Shen[1,†], **Peter Henderson**[1,†]

[1] Princeton University

Link: Project Page | Checkpoint | Code

Abstract. **Mixture-of-Experts (MoE) layers scale model capacity by activating only a sparse subset of expert networks per token. But their routers often switch experts at nearly every step in a rollout. This forecloses memory optimizations, like offloading, once expert counts outgrow GPU capacity. We make the case that this problem is extremely well suited to the options framework with deliberation costs, and argue for *temporally extended MoE* layers. We create a new way to equip each layer with a lightweight controller that learns when to switch expert sets and which to load. By applying this framework to `gpt-oss-20b` with low-rank adapters and a self-distillation reward, our method reduces switch rates from over 50% to below 5% while retaining up to 90% of base-model accuracy on MATH, MMLU, and MMMLU. This shows the surprising viability of the approach. Even existing pre-trained models can be converted to temporally extended MoEs with lightweight training. Temporally extended routing is possible and trade-offs can be governed by deliberation costs. We hope this opens a principled path, grounded in the options framework, for memory-efficient serving and continual learning for ever-growing MoE models.**

1. Introduction

Modern Large Language Models (LLMs) predominantly use some variant of Mixture-of-Experts (MoE) layers in their architecture [32, 9, 3, 19], including `Gemini-2.5-Pro` [11], `GLM5` [12], `Qwen3.5-397B-A17B` [29], `Qwen3-Next-80B-A3B` [30], `DeepSeek-V3` [6], and `gpt-oss` [27]. MoEs activate only a sparse subset of experts for each token, allowing inference-time compute to stay flat even as the total number of parameters grows. So, for example, a 120B parameter model, like `gpt-oss-120b` might only activate 5.1B parameters at a time. In principle, with enough memory, one could add extremely large numbers of experts, while keeping inference latency the same. Scaling the number of experts could potentially come with gains in capabilities [5, 15]. And leveraging growing numbers of experts could even be helpful for improving neuroplasticity and continual learning (though this has yet to be thoroughly explored).

However, this vision becomes challenging with finite memory resources. Once the total number of experts outgrows GPU memory, weights must be offloaded to host memory or disk and loaded on demand [8, 40]. Each load incurs latency that would interrupt workflows and reduce throughput. Current MoE architectures largely ignore this switching cost, assuming that all experts can be kept in memory. Across three frontier open-source MoEs, the average switch rate is large, with the active expert set changing at almost every token generation (§3).

Prior work addresses memory-related challenges for MoEs mainly through two families of approaches. First, some works apply expert pruning to reduce the total number of experts by permanently removing or merging

† Correspondance to: zs7353@princeton.edu, peter.henderson@princeton.edu;
At the moment, for example, RAM costs have grown significantly due to AI-related demand, making scaling memory challenging.

experts, possibly with additional fine-tuning [39, 24, 25]. Second, some works explore caching, prefetching, and offloading-aware serving methods and design heuristics, often based on expert activations or correlations in expert usage across layers or prompts, to decide which experts to keep on GPU and which to fetch from host memory [40, 33, 36, 42].

We observe, however, that this problem has a direct analogue in reinforcement learning. Choosing when to commit to a set of resources and when to pay the cost of switching is exactly the structure formalized by *temporally extended actions* in the options framework [35]. An agent selects a high-level "option" that persists over multiple time steps; switching to a new option incurs a deliberation cost [14].

We propose **temporally extended Mixture-of-Experts**, that train a lightweight controller—a policy over options—to decide when to switch expert sets and which new set to load. The controller is optimized via the option-critic architecture [1] with deliberation costs [14]. Because switching cost is an explicit term in the objective, the controller discovers temporal structure. It switches experts only when the expected quality gain justifies the cost.

In this work we contribute the following. First, we propose the design philosophy of temporally extended MoEs as part of the options framework, identifying that excessive expert switching could lead to missed opportunities in memory optimizations across training, inference, and continual learning. Second, we formalize the dynamic loading of experts as a semi-Markov Decision Process (s-MDP), casting expert masks as options and expert loading latency as a deliberation cost. We adapt and apply the option-critic framework for optimization and design a lightweight controller that can modify most modern MoE architectures. Third, we show that `gpt-oss-20b` can be trained using our option-critic method and a very small amount of adapters to reduce switching rates from over 50% (every other token) to under 5%, or even 1%, with configurable performance trade-offs commensurate with the deliberation cost. This shows that temporally extended MoEs can potentially be leveraged even without large-scale pretraining.

We believe this work points toward a broader principle for MoE post-training. As expert counts continue to grow, potentially scaling with available disk rather than GPU memory, the cost of switching will increasingly dominate serving latency. Training controllers that treat expert loading as temporally extended decisions, with explicit deliberation costs, may offer a principled path for managing this trade-off. We view our framework as a first step in this direction, and present concrete evidence for its viability.

2. Preliminaries

2.1. MDP, s-MDP, and Options

We consider a Markov decision process (MDP) $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$ with states $s \in \mathcal{S}$, actions $a \in \mathcal{A}$, transition kernel $P(s' \mid s, a)$, reward $r(s, a)$, and discount factor $\gamma \in [0, 1)$. A policy $\pi(a \mid s)$ induces a trajectory $\tau = (s_0, a_0, s_1, a_1, \ldots, s_T)$ and a return $G(\tau) = \sum_{t=0}^{T-1} \gamma^t r(s_t, a_t)$.

A semi-Markov decision process (s-MDP) [35] generalizes an MDP by allowing actions that last a variable number of steps: at decision time t_k, the agent picks a high-level action, the environment evolves for a random duration κ_k, and the agent receives the cumulative reward during that duration before making the next decision.

Options are a standard framework to construct such temporally extended actions [35]. An option $\omega \in \Omega$ is defined by a triple $(\mathcal{I}_\omega, \pi_\omega(a \mid s), \beta_\omega(s))$, where $\mathcal{I}_\omega \subseteq \mathcal{S}$ is an initiation set, i.e., states where the option may get started; $\pi_\omega(a \mid s)$ is an intra-option policy over primitive actions; and $\beta_\omega(s) \in [0, 1]$ is a termination function, i.e., the probability that the option terminates upon arriving in state s. A policy over options $\pi_\Omega(\omega \mid s)$ selects which option to start when at state s. We adopt the call-and-return option execution model [1]: An agent starts off with an initial option $\omega_0 \sim \pi_\Omega(\cdot \mid s_0)$. While option ω is active, sample primitive actions

$a_t \sim \pi_\omega(\cdot \mid s_t)$. After each transition to s_{t+1}, terminate the option with probability $\beta_\omega(s_{t+1})$. If the option terminates, we sample a new option $\omega_{t+1} \sim \pi_\Omega(\cdot \mid s_{t+1})$; otherwise, we continue with the same option.

2.2. MoE Routing and Expert Masks

We focus on transformer layers where the MLP block is implemented as an MoE, comprising N experts per MoE layer and L total layers. For token position t in layer ℓ, the MoE router produces logits $g_t^{(\ell)} \in \mathbb{R}^N$ from which a distribution is obtained by

$$p_t^{(\ell)} = \text{softmax}(g_t^{(\ell)}).$$

In the base MoE router, a sparse top-$\tilde{k}$ set of experts is selected based on $g_t^{(\ell)}$, and the expert outputs are combined with normalized routing weights $p_t^{(\ell)}$. In our setting, routing is additionally constrained by a binary expert mask $\omega_t^{(\ell)} \in \{0,1\}^N$, where $\omega_{t,i}^{(\ell)} = 1$ indicates that expert i is allowed at time t in layer ℓ. The top-$\tilde{k}$ expert selection is restricted to the set of allowed experts. This binary expert mask $\omega_t^{(\ell)}$ is the option in our setting. Throughout the paper, we use $\hat{k}$ to denote the number of allowed experts by the expert mask, and $\tilde{k}$ to denote the number of activated experts.

We say a **switch** occurs at token position t in layer ℓ whenever $\omega_t^{(\ell)} \neq \omega_{t-1}^{(\ell)}$, i.e., when the expert mask changes. In the options framework, this corresponds to the termination of the current option and the selection of a new one. The **switch rate** of a generated sequence of length T is

$$\frac{1}{L}\sum_{\ell=1}^{L}\frac{1}{T-1}\sum_{t=1}^{T-1}\mathbf{1}\left[\omega_t^{(\ell)} \neq \omega_{t-1}^{(\ell)}\right].$$

With a learned controller (Section 4), the controller selects $\omega_t^{(\ell)}$ at each position. For the base model, we define switching rates by treating the router itself as an option-selection policy: at each switch point, $\omega_t^{(\ell)}$ is set to the top-$\hat{k}$ experts according to $g_t^{(\ell)}$, and this mask persists until the $\tilde{k}$ activated experts at some future position are no longer fully contained in $\omega^{(\ell)}$, triggering a new switch. The base model switch rate thus measures how often the router's own selections would necessitate reloading experts, providing a reference point that our learned controller aims to reduce.

2.3. Option-Critic Architecture

The option-critic architecture [1] extends the policy gradient theorem to the options framework and allows for optimizing both the intra-option policies π_ω and the termination functions β_ω simultaneously. To derive the gradients, we first define the value of executing an action a under state-option pair (s, ω) as

$$Q_U(s,\omega,a) = r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) U(\omega, s'),$$

where $U(\omega, s')$ is the value of being in state s' with option ω currently active:

$$U(\omega,s') = (1-\beta_\omega(s'))Q_\Omega(s',\omega) + \beta_\omega(s')V_\Omega(s').$$

Here, $Q_\Omega(s,\omega) = \sum_a \pi_\omega(a \mid s) Q_U(s,\omega,a)$ is the value of executing option ω starting in state s, and $V_\Omega(s) = \sum_\omega \pi_\Omega(\omega \mid s) Q_\Omega(s,\omega)$ is the value of being in state s.

The option-critic framework optimizes the parameters of the intra-option policies π_ω (denoted by θ) and the termination functions β_ω (denoted by ν) to maximize the expected discounted return. This is realized via the following two theorems:

Theorem 1 (Intra-Option Policy Gradient Theorem, Theorem 1 of [1])**.** *Given a set of options with stochastic intra-option policies $\pi_{\omega,\theta}$ differentiable with respect to θ, the gradient of the expected discounted return with respect to θ is*

$$\frac{\partial Q_\Omega(s_0,\omega_0)}{\partial\theta} = \sum_{s,\omega}\mu(s,\omega)\sum_a \frac{\partial \pi_\omega(a \mid s)}{\partial\theta} Q_U(s,\omega,a),$$

where $\mu(s,\omega)$ is a discounted weighting of state-option pairs along trajectories starting from (s_0,ω_0).

Theorem 2 (Termination Gradient Theorem, Theorem 2 of [1])**.** *Given a set of options with stochastic termination functions $\beta_{\omega,\nu}$ differentiable with respect to ν, the gradient of the expected discounted return with respect to ν is*

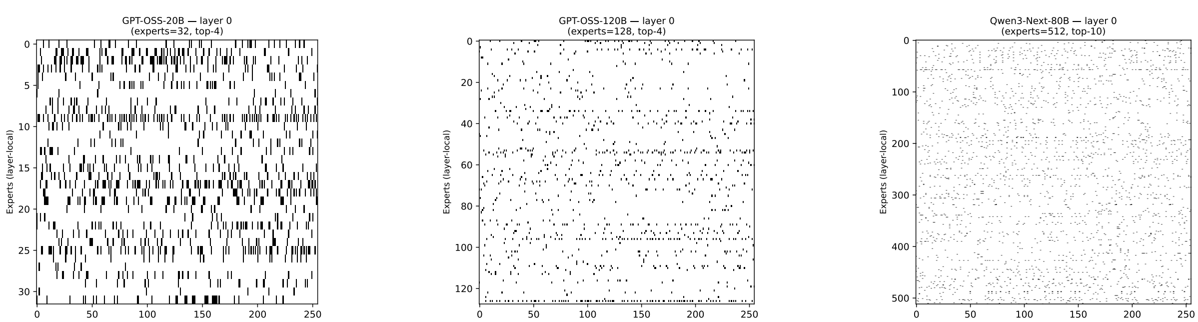
$$\frac{\partial Q_\Omega(s_0,\omega_0)}{\partial\nu} = -\sum_{s,\omega}\mu(s,\omega)\frac{\partial \beta_\omega(s)}{\partial\nu}\left(Q_\Omega(s,\omega) - V_\Omega(s)\right).$$

Intuitively, $Q_\Omega(s,\omega) - V_\Omega(s)$ is the advantage of the current option relative to the expected value of switching to a new option. If the current option offers a higher value than the expected value of a newly selected option, the gradient update decreases termination probability $\beta_\omega(s)$, thereby extending the current option's duration.

3. Motivation

3.1. Current Mixture-of-Experts LLMs are not Temporally Extended

We first show that current MoE models are not temporally extended. We measure the switch rate of three frontier open-source MoE models — `gpt-oss-20b` (32 experts, top-4), `gpt-oss-120b` (128 experts, top-4), and `Qwen3-Next-80B-A3B` (512 experts, top-10) — on 100 prompts from each of the 10 categories in the Nemotron Post-Training Dataset v2 [26]. For each prompt, we generate 256 tokens with temperature 0.5 and record which experts are activated at every token position and every layer.



(a) gpt-oss-20b (b) gpt-oss-120b (c) Qwen3-Next-80B-A3B

Fig. 1: Set of activated experts in layer 0 throughout the trajectory for each model

Figure 1 visualizes the expert activation pattern at layer 0 for an arbitrarily selected prompt, where the x-axis is the token position and the y-axis are the experts. We can see that, across all three models, expert selection shows little temporal continuity, confirmed in Table 1. The average switch rate is close to 1 for all models, switching at nearly every token.

Model	Chat	Code	Math	STEM	Multi (en)	Multi (de)	Multi (es)	Multi (fr)	Multi (it)	Multi (ja)
gpt-oss-20b	$0.94_{\pm 0.06}$	$0.95_{\pm 0.01}$	$0.94_{\pm 0.02}$	$0.95_{\pm 0.01}$	$0.95_{\pm 0.02}$	$0.95_{\pm 0.01}$	$0.95_{\pm 0.02}$	$0.95_{\pm 0.02}$	$0.95_{\pm 0.02}$	$0.95_{\pm 0.01}$
gpt-oss-120b	$0.98_{\pm 0.01}$	$0.99_{\pm 0.00}$	$0.99_{\pm 0.00}$	$0.99_{\pm 0.00}$	$0.99_{\pm 0.00}$	$0.99_{\pm 0.00}$	$0.99_{\pm 0.00}$	$0.99_{\pm 0.00}$	$0.99_{\pm 0.00}$	$0.99_{\pm 0.00}$
Qwen3-Next-80B	$1.00_{\pm 0.00}$	$1.00_{\pm 0.00}$	$1.00_{\pm 0.00}$	$1.00_{\pm 0.00}$	$1.00_{\pm 0.00}$	$1.00_{\pm 0.00}$	$1.00_{\pm 0.00}$	$1.00_{\pm 0.00}$	$1.00_{\pm 0.00}$	$1.00_{\pm 0.00}$

Table 1: Average switch rate (mean $\pm$ std) with $\hat{k} = \tilde{k}$ across 100 prompts per category.

3.2. Missed Opportunities

The lack of temporal extension in current MoE routing leads to missed optimization opportunities across the model life-cycle. We highlight three such opportunities that temporally extended expert selection can unlock.

Inference serving with reduced memory. During autoregressive generation, standard MoE serving must keep all N experts per layer resident in fast device memory (or be prepared to fetch any of them at each step), because the set of active experts is not temporally extended. When expert weights do not fit on the available GPUs, systems resort to offloading experts to host memory and prefetching them as needed [40, 33], but mispredictions can lead to significant latency. With temporal continuity, the active expert set is known to persist for many consecutive tokens, reducing costs from prefetching misses and enabling a simpler and more predictable serving strategy. Only the $\hat{k}$ active experts per layer need to reside on the GPU, and expert swaps occur only occasionally. Between switches, inference proceeds with $\frac{\hat{k}}{N}$ of the expert memory footprint. Since expert parameters dominate the total parameter count in modern MoE models (e.g., over 96% in `gpt-oss-20b`), this directly translates to a substantial reduction in GPU memory requirements for serving. For example, keeping only 16 experts in $\hat{k}$ for `gpt-oss-20b` reduces VRAM requirements by $\sim$4.7 GiB (37%) for 16 experts, or $\sim$7.1 GiB (55%) for 8 experts.

Memory-efficient training via temporal chunking. A similar principle applies during training. In current MoE training pipelines, all expert parameters must be accessible during the forward and backward pass, since any token in the sequence may route to any expert. With temporally extended routing, a response can be partitioned into contiguous chunks, each associated with a fixed expert mask. Within each chunk, only the $\hat{k}$ experts in the current mask participate in the forward and backward computation. This opens the door to chunk-wise training strategies where inactive experts are offloaded during each chunk's forward-backward pass, reducing peak GPU memory.

Continual learning with expandable expert capacity. Temporal extension also offers a natural path toward continual learning. Because only $\hat{k}$ out of N experts are active at any time, new experts can be added to the model without increasing the per-token compute or the active memory footprint. When adapting to a new domain or task, one can initialize fresh expert modules and let the controller learn to route to them when beneficial. The fixed active set size ($\hat{k}$) ensures that inference cost remains constant regardless of the new experts you added.

4. Method

In this section, we present our method towards temporally-extended control of expert routing in MoE transformers. As noted earlier, we maintain a per-layer option $\omega_t^{(\ell)}$ — an expert mask for the allowed subset of $\hat{k}$ experts — and restrict the router to select only from this set. We implement a lightweight controller that learns when to switch and which new expert mask to switch to, while simultaneously fine-tuning the MoE model's parameters via intra-option policy update.

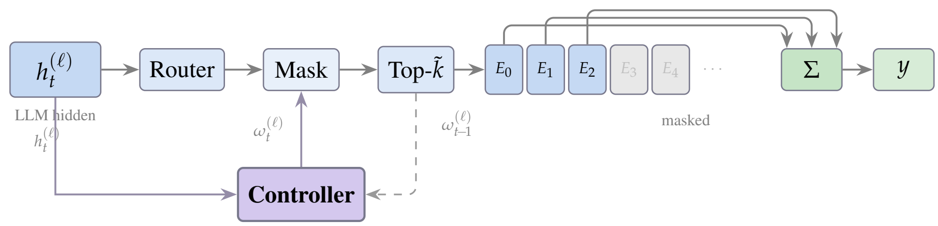


Fig. 2: Controller integration with MoE layer. The controller observes LLM hidden states $h_t^{(\ell)}$ and the current option $\omega_{t-1}^{(\ell)}$, and outputs a new expert mask $\omega_t^{(\ell)}$ that restricts which experts can be selected by top-$\tilde{k}$ routing. Grayed experts are masked out.

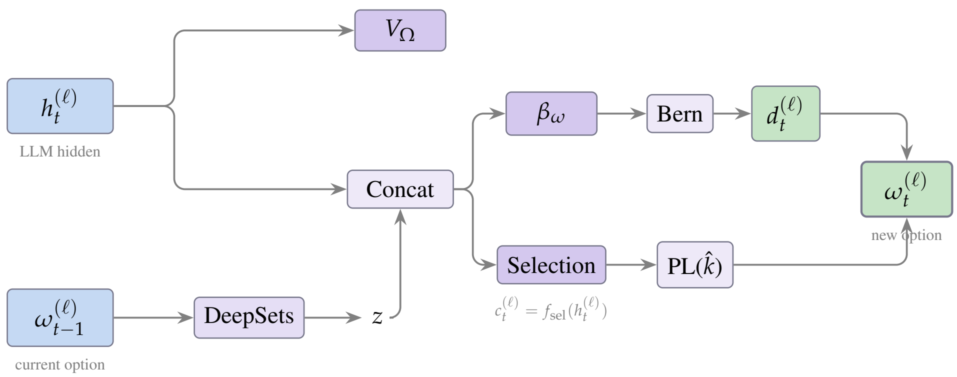


Fig. 3: Controller architecture. The controller takes LLM hidden states $h_t^{(\ell)}$ and the current option $\omega_{t-1}^{(\ell)}$ (a set of $\hat{k}$ expert indices) as inputs. A DeepSets encoder maps $\omega_{t-1}^{(\ell)}$ to a permutation-invariant set embedding z, which is concatenated with $h_t^{(\ell)}$ (after RMSNorm) and fed to the termination head β_ω and the selection head. The state-value head V_Ω uses $h_t^{(\ell)}$ alone. The selection head is a linear layer initialized from the router weights and produces candidate logits $c_t^{(\ell)} = f_{\text{sel}}(h_t^{(\ell)})$, from which $\hat{k}$ experts are sampled via the Plackett–Luce distribution. If the Bernoulli sample $d_t^{(\ell)} = 1$, the newly sampled option replaces the current one; otherwise the current option persists.

4.1. Options Formulation for Expert Mask Control

For each layer ℓ, the space of options is the combination of possible masks over experts:

$$\Omega^{(\ell)} = \left\{ \omega \in \{0,1\}^N : \|\omega\|_1 = \hat{k} \right\}. \tag{1}$$

Our controller implements call-and-return execution, an active option $\omega^{(\ell)}$ persists across tokens until a termination decision $d_t^{(\ell)} = 1$ triggers selection of a new option. During execution, the router is constrained to select only from the experts in the active option by masking all other experts' logits to $-\infty$ before the top-$\tilde{k}$ operation. In principle, one could treat the joint mask $\omega_t = (\omega_t^{(1)}, \ldots, \omega_t^{(L)})$ as a single option and learn a joint termination and selection policy over all layers. We instead factorize the controller into L independent per-layer controllers for tractability, each conditioning on its layer hidden state and current mask. This yields an approximation to the joint s-MDP: each layer's controller treats the rest of the network as part of the environment while sharing the same token-level reward. Despite this approximation, it is stable to train and achieves both good performance and large switch rate reductions in practice.

4.2. Controller Architecture

Every MoE MLP layer ℓ has its own controller module, with the same architecture but separate parameters. At token t and layer ℓ, recall that $p_t^{(\ell)}$ is the softmax of the router logits, and $\omega_{t-1}^{(\ell)}$ is the active expert mask from the previous step. Our controller operates directly on the LLM pre-MLP hidden representation $h_t^{(\ell)}$, i.e., treat $h_t^{(\ell)}$ as the state s. In the following, we walk through the major components of our controller.

Expert set embedding. Recall that each option ω is an expert mask. To obtain a richer representation of expert masks, we use a DeepSets encoder [41]:

$$z^{(\ell)}(\omega) = \frac{1}{\hat{k}} \sum_{i \in \omega} \varphi(e_i), \tag{2}$$

where $e_i \in \mathbb{R}^{d_e}$ is a learned embedding for expert i and $\varphi : \mathbb{R}^{d_e} \to \mathbb{R}^{d_c}$ is a two-layer MLP with GELU activation. Each layer has a separate encoder.

Termination head. The termination decision $\beta_t^{(\ell)}$ depends on both the LLM state $h_t^{(\ell)}$ and the current option $\omega_{t-1}^{(\ell)}$. We compute

$$\beta_t^{(\ell)} = \sigma\left(\mathrm{MLP}_\beta\left(\mathrm{concat}\left(\overline{h}_t^{(\ell)}, \overline{z}^{(\ell)}(\omega_{t-1}^{(\ell)})\right)\right)\right), \tag{3}$$

where $\overline{h}_t^{(\ell)} = \mathrm{RMSNorm}(h_t^{(\ell)})$ and $\overline{z}^{(\ell)} = \mathrm{RMSNorm}(z^{(\ell)})$ balance the scale of the two representations, MLP_β is a two-layer MLP with ReLU activation, and σ is the sigmoid function. A switch decision is then sampled as $d_t^{(\ell)} \sim \mathrm{Bernoulli}(\beta_t^{(\ell)})$.

Value and option-value heads. The state-value function $V_\Omega(h_t^{(\ell)}) = w_V^\top h_t^{(\ell)} + b_V$ is a linear head on the LLM hidden states. The option-value function is

$$Q_\Omega(h_t^{(\ell)}, \omega) = \mathrm{MLP}_Q\left(\mathrm{concat}\left(\overline{h}_t^{(\ell)}, \overline{z}^{(\ell)}(\omega)\right)\right), \tag{4}$$

where $\overline{h}_t^{(\ell)} = \mathrm{RMSNorm}(h_t^{(\ell)})$, $\overline{z}^{(\ell)} = \mathrm{RMSNorm}(z^{(\ell)})$, and MLP_Q is a two-layer MLP with ReLU activation.

Option selection head. When $d_t^{(\ell)} = 1$, a new option must be selected. We use a selection head $f_{\mathrm{sel}}^{(\ell)} : \mathbb{R}^d \to \mathbb{R}^N$, a linear layer initialized from the router weights, to produce candidate logits $c_t^{(\ell)} = f_{\mathrm{sel}}(h_t^{(\ell)})$. We sample $\hat{k}$ experts from the Plackett-Luce (PL) distribution, which defines a probability over ordered selections $(i_1, \ldots, i_{\hat{k}})$ by sequentially sampling without replacement:

$$P_{\mathrm{PL}}(i_1, \ldots, i_{\hat{k}} \mid c) = \prod_{j=1}^{\hat{k}} \frac{\exp(c_{i_j})}{\sum_{m \notin \{i_1, \ldots, i_{j-1}\}} \exp(c_m)}. \tag{5}$$

We denote the induced distribution as $\pi_{\mathrm{sel}}(\omega \mid h)$. In real implementation, sampling sequentially is slow, so we perform sampling via the Gumbel-top-$\hat{k}$ trick: we add i.i.d. $\mathrm{Gumbel}(0, 1)$ noise to $c_t^{(\ell)}$ and take the top-$\hat{k}$ indices of the perturbed logits, which is mathematically equivalent but fully vectorized. The new option is $\omega_t^{(\ell)} = \{i_1, \ldots, i_{\hat{k}}\}$. Note that we treat the sampled top-$\hat{k}$ indices as an ordered tuple distributed by PL; the mask is the induced unordered set, but the policy-gradient uses the ordered PL log-prob of the sampled tuple.

Initialization. At $t = 0$, the initial option is set to the top-$\hat{k}$ experts under the router logits. The switch decision is forced to $d_0^{(\ell)} = 0$.

4.3. Controller Training

We treat the non-controller parameters within the MoE model as intra-option policy. We train the controller and the MoE model using the option-critic with deliberation cost framework [14] with per-token dense rewards.

Reward design. In our case, our goal is to transform a pretrained MoE into a temporally-extended MoE while maintaining its previous quality and performance. As such we follow [23]. We use the per-token reverse KL, the divergence between the student's and teacher's distribution for each token conditioned on the same prior trajectory, as the per-token reward. The teacher is the original frozen MoE model (without controller and any weight updates). The student is the model we train. Specifically,

$$r_t = \log p_{\text{teacher}}(a_t \mid x, a_{<t}) - \log p_{\text{student}}(a_t \mid x, a_{<t}), \tag{6}$$

where a_t is the token generated at step t. Note that in expectation over the student's sampling distribution, $-r_t$ is an unbiased estimator of the reverse KL divergence $\mathrm{KL}(p_{\text{student}} \| p_{\text{teacher}})$.

To avoid reward hacking where the trained model generates degenerate and repetitive outputs with low reverse KL, we sample tokens from a mixture of the student and teacher distributions following [13]:

$$p_{\text{mix}} = (1 - \tau)\, p_{\text{student}} + \tau\, p_{\text{teacher}}, \tag{7}$$

with approximate importance weights $w_t = \frac{p_{\text{student}}(a_t|x, a_{<t})}{p_{\text{mix}}(a_t|x, a_{<t})}$ applied in the policy gradient to correct for the off-policy sampling. Note that even though this is a biased approximation of the importance weights (the importance weights should have been $\prod_{t'=1}^{t} \frac{p_{\text{student}}(a_{t'}|x, a_{<t'})}{p_{\text{mix}}(a_{t'}|x, a_{<t'})}$), we still use this approximation because [13] found that it works better as it reduces the variance of gradient computation. In cases where a model is being post-trained from scratch, rather than being self-distilled into a temporally extended MoE, these rewards can be swapped for standard post-training rewards.

Gradient updates. Our training procedure follows the A2OC algorithm of [14] adapted to our setting. We describe how each gradient update is instantiated.

For critic learning, we learn V_Ω and Q_Ω by minimizing squared TD errors with GAE(λ) [31] targets. For V_Ω, the TD error at token t and layer ℓ is $\delta_t^V = r_t + \gamma V_\Omega(h_{t+1}^{(\ell)}) - V_\Omega(h_t^{(\ell)})$. For Q_Ω, we bootstrap with $U(\omega, s')$ as defined in Section 2: $\delta_t^Q = r_t + \gamma U(\omega_t^{(\ell)}, h_{t+1}^{(\ell)}) - Q_\Omega(h_t^{(\ell)}, \omega_t^{(\ell)})$, where $\omega_t^{(\ell)}$ is the option executed at step t. The critics jointly minimize $\left(V_\Omega(h_t^{(\ell)}) - \hat{V}_t^{\text{targ}}\right)^2 + \left(Q_\Omega(h_t^{(\ell)}, \omega_t^{(\ell)}) - \hat{Q}_t^{\text{targ}}\right)^2$ respectively, with targets $\hat{V}_t^{\text{targ}} = V_\Omega(h_t^{(\ell)}) + \hat{A}_t^V$ and $\hat{Q}_t^{\text{targ}} = Q_\Omega(h_t^{(\ell)}, \omega_t^{(\ell)}) + \hat{A}_t^Q$ computed from the GAE advantages $\hat{A}_t^V = \sum_{j=0}^{\infty} (\gamma\lambda)^j \delta_{t+j}^V$ and $\hat{A}_t^Q = \sum_{j=0}^{\infty} (\gamma\lambda)^j \delta_{t+j}^Q$.

For intra-option policy update, we apply Theorem 1 to update the intra-option policy parameters θ (expert and router parameters). In our setting, the intra-option policy π_ω is the LLM next-token-probability distribution and the primitive actions are generated tokens. Using log-derivative trick, the gradient from Theorem 1 can be written as

$$\mathbb{E}_{(s,\omega)\sim\mu,\, a\sim\pi_{\omega,\theta}} \left[\frac{\partial \log \pi_\omega(a \mid s)}{\partial \theta} Q_U(s, \omega, a) \right]. \tag{8}$$

In practice, we estimate $Q_U(s, \omega, a)$ with the Monte Carlo return $\bar{G}_t = \sum_{j \geq 0} \gamma^j r_{t+j}$.

For termination gradient update, we augment Theorem 2 with the deliberation cost η as in [14]. The gradient of the expected return with respect to the termination parameters ν becomes

$$-\sum_{s,\omega}\mu(s,\omega)\frac{\partial\beta_\omega(s)}{\partial\nu}\Big(Q_\Omega(s,\omega)-V_\Omega(s)+\eta\Big), \tag{9}$$

where $\mu(s,\omega)$ is the discounted state-option visitation distribution as defined in Theorem 1. Here, η serves as a margin so that termination is only preferred when the current option is sufficiently worse than alternatives to overcome the deliberation cost.

For the option selection heads, we update them only when a switch occurs. In particular, when $d_t^{(\ell)}=1$, we update the option selection head parameters ϕ using the policy gradient

$$\sum_{s,\omega}\mu(s,\omega)\nabla_\phi \log\pi_{\text{sel}}(\omega\mid h)\left(Q_\Omega(s,\omega)-V_\Omega(s)\right). \tag{10}$$

We present a high-level version of the algorithm in Algorithm 1 and the full algorithm as well as additional implementation details in Section A2.

5. Experiments

In this section, we present our main experimental setup and results.

Training details. We conduct all experiments on `gpt-oss-20b` [27], a Mixture-of-Experts LLM with 24 transformer layers, 32 experts per layer, and top-4 routing ($\tilde{k}=4$). The model uses MXFP4 quantization natively and is dequantized to bf16 for training. We train on 4× NVIDIA 140GB H200 GPUs, using a modified version of the TRL library [37].

For training, we use the following hyperparameters: discount factor $\gamma=0.95$, GAE parameter $\lambda=0.95$, value loss coefficient 0.01. The controller is trained with learning rates $\alpha_{\text{controller}}=10^{-4}$ using AdamW. For the intra-option policy update, we apply LoRA [18] with rank $r=16$ and $\alpha=16$ to both expert parameters and attention parameters. Router weights are also trainable. The intra-option policy learning rate is $\alpha_{\text{intra}}=2\times10^{-4}$. Each training step uses 16 prompts with a maximum prompt length of 512 tokens and max response length of 512 tokens. Token generation uses temperature 1.0 and Top-$p=0.95$. The teacher mixing ratio is $\tau=0.2$.

Datasets and benchmarks. For training, we use the Nemotron Post-Training Dataset v2 [26], which contains prompts across 10 categories: chat, code, math, STEM, and multilingual (English, German, Spanish, French, Italian, Japanese). We use all categories. For evaluation, we evaluate on 200 randomly selected questions from MATH dataset [17], MMLU, and MMMLU [16], respectively. For MATH, we check correctness using the `is_equiv()` function provided in the official github repository of [17]. All evaluations use temperature 0.5, Top-$p=0.95$, and max response length of 2048 tokens. Random seed is 42 across all experiments.

Baselines. We compare our trained controller with four pruning baselines: frequency-based selection, reconstruction loss minimization [24], random selection, and Wanda (structured) [34]. For all baselines, we use 128 prompts randomly drawn from the Nemotron Post-training Dataset v2 as the calibration set—**(author?)** [24] found 128 sequences to be best for calibration. For each prompt, we generate a response with `gpt-oss-20b`

(author?) [24] use 128 text pieces drawn from C4 pretraining corpus (without any model generated responses) as the calibration set, which we found to have very poor performance in our setting, so we switch to Nemotron Post-training Dataset v2 and include model generated responses to have a meaningful comparison.

Algorithm 1: MoE Option-Critic Training (High-Level)

Input: MoE model with L layers, N experts per layer, top-$\tilde{k}$ routing; prompt dataset $\mathcal{D}$; teacher model p_{teacher}; teacher mixing coefficient τ; discount γ; GAE parameter λ; deliberation cost η, learning rates $\alpha_{\text{controller}}$, α_{intra}. We use θ to denote the LLM parameters, ν to denote the termination head, ψ to denote the critic parameters (including V_Ω and Q_Ω), and ϕ to denote the option selection head parameters.

for *each training iteration* **do**
  Sample prompt $x \sim \mathcal{D}$;
  `// Rollout with teacher mixing`
  Initialise $\omega_0^{(\ell)} \leftarrow \mathrm{TopK}(g_0^{(\ell)}, \hat{k})$ from router logits, for each layer ℓ;
  for $t = 1, \ldots, T$ **do**
    for *each layer* ℓ **do**
      Compute termination probability $\beta_t^{(\ell)} \in [0,1]$ from hidden state $h_t^{(\ell)}$ and current option $\omega_{t-1}^{(\ell)}$;
      Sample $d_t^{(\ell)} \sim \mathrm{Bernoulli}(\beta_t^{(\ell)})$;
      if $d_t^{(\ell)} = 1$ **then** select new option $\omega_t^{(\ell)}$ by sampling $\hat{k}$ experts via Plackett–Luce;
      else persist $\omega_t^{(\ell)} \leftarrow \omega_{t-1}^{(\ell)}$;
      Mask router to experts in $\omega_t^{(\ell)}$;
    Sample token $a_t \sim p_{\text{mix}} = (1-\tau)\,\pi_{\omega,\theta} + \tau\, p_{\text{teacher}}$;
    Record importance weight $w_t \leftarrow \pi_{\omega,\theta}(a_t) / p_{\text{mix}}(a_t)$ and reward $r_t \leftarrow \log p_{\text{teacher}}(a_t) - \log \pi_{\omega,\theta}(a_t)$;
  `// Controller update`
  for *each layer* ℓ **do**
    Compute GAE(λ) targets $\hat{V}_t^{\text{targ}}$, $\hat{Q}_t^{\text{targ}}$ using V_Ω, Q_Ω, and r_t;
    for $t = 1, \ldots, T$ **do**
      Accumulate termination gradient: $d\nu \mathrel{-}= w_t \nabla_\nu \beta_t^{(\ell)} \big(Q_\Omega(h_t^{(\ell)}, \omega_{t-1}^{(\ell)}) - V_\Omega(h_t^{(\ell)}) + \eta\big)$;
      Accumulate selection gradient (when $d_t^{(\ell)} = 1$): $d\phi \mathrel{+}= w_t \nabla_\phi \log \pi_{\text{sel}}(\omega_t^{(\ell)}) \big(Q_\Omega(h_t^{(\ell)}, \omega_t^{(\ell)}) - V_\Omega(h_t^{(\ell)})\big)$;
      Accumulate critic loss: $d\psi \mathrel{-}= \nabla_\psi \big[(V_\Omega - \hat{V}_t^{\text{targ}})^2 + (Q_\Omega - \hat{Q}_t^{\text{targ}})^2\big]$;
  `// Intra-option policy update`
  for $t = 1, \ldots, T$ **do**
    Compute discounted return $\bar{G}_t = \sum_{j \geq 0} \gamma^j r_{t+j}$;
    $d\theta \mathrel{+}= w_t \nabla_\theta \log \pi_{\omega,\theta}(a_t) \cdot \bar{G}_t$;
  $(\nu, \psi, \phi) \leftarrow (\nu, \psi, \phi) + \alpha_{\text{ctrl}} \frac{1}{L} (d\nu, d\psi, d\phi)$; $\theta \leftarrow \theta + \alpha_{\text{intra}}\, d\theta$;

so that the prompt length plus the response length does not exceed 2048. All other configurations follow [24]. For frequency-based selection, we keep the $\hat{k}$ experts that are most frequently used on the calibration set. For reconstruction loss minimization, we keep a set of $\hat{k}$ experts that minimizes a reconstruction loss. We provide additional details about our implementation of reconstruction loss minimization in Section A2. For random selection, we randomly select a set of $\hat{k}$ experts at each token. For Wanda (structured), we conduct structured weight pruning and prune out $\frac{N-\hat{k}}{N}$ of the weights following [34].

Benchmark	Base Model	Pruning Baselines				Ours (Learned Controller)		
		Frequency	Reconstruction	Random	Wanda	η=0.02	η=0.03	η=0.04
MATH	$71.5_{\pm 5.9}$	$53.5_{\pm 6.9}$	$51.5_{\pm 6.9}$	$15.0_{\pm 4.9}$	$3.5_{\pm 2.5}$	$\mathbf{64.0}_{\pm 6.7}$	$58.5_{\pm 6.9}$	$55.0_{\pm 6.9}$
switch %	$58.6_{\pm 0.51}$					$4.1_{\pm 0.02}$	$1.3_{\pm 0.02}$	$1.2_{\pm 0.02}$
MMLU	$79.5_{\pm 5.7}$	$55.5_{\pm 6.9}$	$35.0_{\pm 6.7}$	$33.5_{\pm 6.5}$	$9.0_{\pm 3.9}$	$\mathbf{72.5}_{\pm 6.3}$	$67.5_{\pm 6.5}$	$63.0_{\pm 6.7}$
switch %	$57.1_{\pm 0.53}$					$4.2_{\pm 0.02}$	$1.3_{\pm 0.02}$	$1.2_{\pm 0.02}$
MMMLU	$67.5_{\pm 6.5}$	$42.0_{\pm 6.9}$	$48.0_{\pm 6.9}$	$24.0_{\pm 5.9}$	$7.0_{\pm 3.5}$	$\mathbf{59.5}_{\pm 6.9}$	$56.5_{\pm 6.9}$	$49.5_{\pm 6.9}$
switch %	$54.5_{\pm 0.51}$					$4.2_{\pm 0.02}$	$1.4_{\pm 0.02}$	$1.2_{\pm 0.02}$

Table 2: Accuracy (%, mean $\pm$ 95% CI) and switch rate (%, mean $\pm$ 95% CI) with $\hat{k} = 16$. "Ours" denotes our trained controller with deliberation cost η.

Benchmark	Base Model	Pruning Baselines				Ours (Learned Controller)		
		Frequency	Reconstruction	Random	Wanda	η=0.02	η=0.03	η=0.04
MATH	$71.5_{\pm 5.9}$	$11.5_{\pm 4.3}$	$7.5_{\pm 3.5}$	$0.0_{\pm 0.0}$	$0.0_{\pm 0.0}$	$\mathbf{27.5}_{\pm 6.1}$	$23.0_{\pm 5.9}$	$15.5_{\pm 4.9}$
switch %	$79.0_{\pm 0.39}$					$9.2_{\pm 0.14}$	$7.4_{\pm 0.12}$	$5.4_{\pm 0.10}$
MMLU	$79.5_{\pm 5.7}$	$12.5_{\pm 4.5}$	$2.5_{\pm 2.2}$	$4.0_{\pm 2.7}$	$0.0_{\pm 0.0}$	$\mathbf{48.5}_{\pm 6.9}$	$41.0_{\pm 6.9}$	$38.0_{\pm 6.7}$
switch %	$77.4_{\pm 0.45}$					$8.5_{\pm 0.10}$	$7.6_{\pm 0.08}$	$5.0_{\pm 0.06}$
MMMLU	$67.5_{\pm 6.5}$	$8.5_{\pm 3.9}$	$1.0_{\pm 1.4}$	$3.0_{\pm 2.4}$	$0.0_{\pm 0.0}$	$\mathbf{39.0}_{\pm 6.5}$	$31.5_{\pm 6.3}$	$22.5_{\pm 5.9}$
switch %	$75.5_{\pm 0.43}$					$9.0_{\pm 0.14}$	$8.0_{\pm 0.10}$	$5.4_{\pm 0.08}$

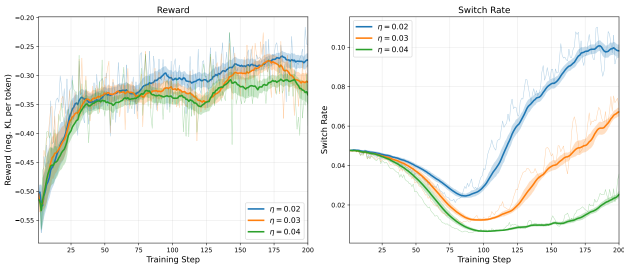
Table 3: Accuracy (%, mean $\pm$ 95% CI) and switch rate (%, mean $\pm$ 95% CI) with $\hat{k} = 8$. "Ours" denotes our trained controller with deliberation cost η.

Training Dynamics. We train the controller with varying deliberation costs $\eta \in \{0.02, 0.03, 0.04\}$ and expert budgets $\hat{k} \in \{8, 16\}$. The training curves of our runs are presented in Figure 4. Across different configurations, the reward steadily increases during training, with more pronounced gains under $\hat{k} = 8$. The switch rate initially decreases (as the value networks V_Ω, Q_Ω are learning) and gradually stabilizes at a level determined by η, with higher deliberation costs yielding lower converged switch rates. Perplexity also decreases throughout training, with clearer improvements for $\hat{k} = 8$.

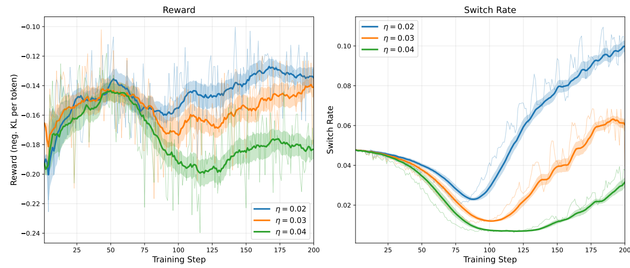
Benchmark Evaluation. For $\hat{k} = 8$, we evaluate the checkpoint at step 300. For $\hat{k} = 16$, we evaluate the checkpoint at step 120. With $\hat{k} = 16$ and $\eta = 0.02$, our controller achieves accuracy close to the unpruned

We do not compare with caching and prefetching approaches such as MoE-infinity [40], because they optimize the memory management of expert weights during inference rather than alter the model's routing decisions. Every expert selected by the router is still computed, with cache misses only incurring additional latency. This is different from our setting where we want to study utility-efficiency tradeoff. In fact, **(author?)** [40]'s work can directly be used with our method to allow for off-loading and on-loading of experts when switching options.

Since we report a single training run per configuration, the observed variance in the curves reflects within-run stochasticity rather than across-run stability; we do not claim one setting trains more stably than another, only that we are able to yield a well-performing temporally extended MoE in at least one run.



(a) Training curves for $\hat{k} = 8$.

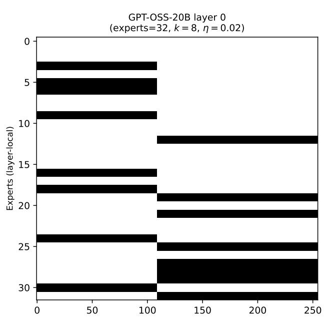


(b) Training curves for $\hat{k} = 16$

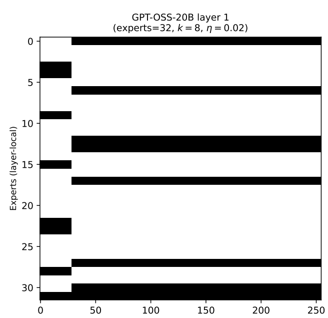
Fig. 4: Reward curves and switch rate curves for one training run. We present the running averages over sliding windows of size 20. The shaded bands show bootstrap 95% confidence intervals for the running mean, computed by resampling (with replacement, 1000 times) within a sliding window of 20 steps and taking the 2.5th and 97.5th percentiles of the resampled means.

base model and substantially outperforms all baselines across benchmarks. Performance shows a trade-off in performance commensurate with the deliberation cost and the size of the mask $\hat{k}$. This trade-off might improve with a full post-training run and can be calibrated with the deliberation cost.

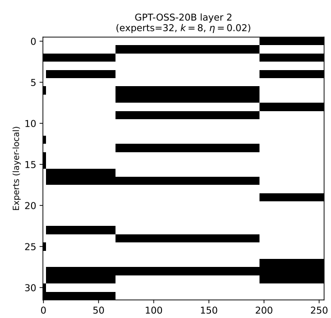
Temporal continuity with controller. As a direct comparison with Figure 1, we plot the option (i.e., expert mask) of `gpt-oss-20b` under our trained controller (with $\eta = 0.02$) for the same prompt used in Section 3 throughout the generated trajectory in layer 0, 1, 2, respectively, in Figures 5 and 6, where x-axis is the token position and y-axis are the experts. We still generate 256 tokens with temperature 0.5 and record the expert mask at every token position and every layer. We can see that the expert selection shows significantly stronger temporal continuity. Note that different layers can have different levels of temporal continuity. We present additional plots under controller trained with $\eta = 0.03, 0.04$ in Section A3, which show similar patterns.



(a) Layer 0



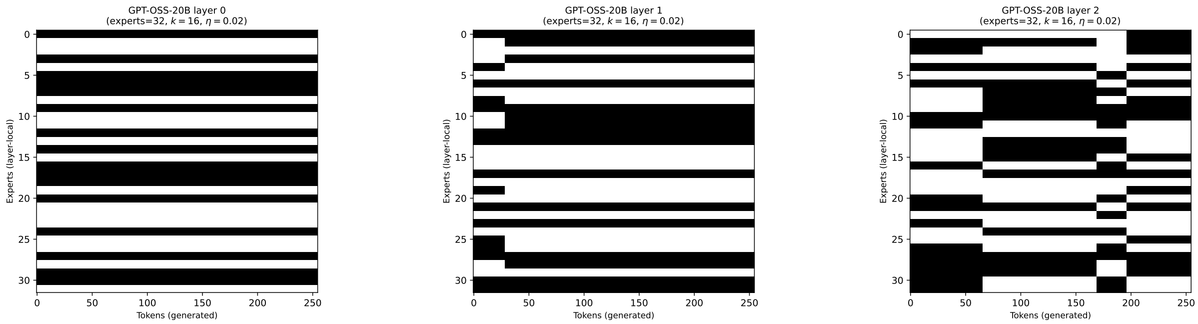
(b) Layer 1



(c) Layer 2

Fig. 5: Options in layer 0, 1, 2 throughout the trajectory with gpt-oss-20b, $\hat{k} = 8, \eta = 0.02$.

We present additional analysis of training and results, including loss curves, additional switch rate analysis, repetition and perplexity analysis, in Section A3, and concrete examples of model responses with different methods in Section A5.



(a) Layer 0 (b) Layer 1 (c) Layer 2

Fig. 6: Options in layer 0, 1, 2 throughout the trajectory with gpt-oss-20b, $\hat{k} = 16, \eta = 0.02$.

6. Discussion and Conclusion

In this work, we introduced the concept of temporally extended MoE models and presented a framework that addresses dynamic expert loading using the options framework. Our method effectively balances generation quality with the latency cost of expert set transfer. Our findings also point to a promising future direction of designing MoE architectures in a temporally extended way by making it a core objective during post-training, and potentially even during pre-training. Developing such inherently temporally extended MoE models could minimize expert switches by design, further closing the gap between massive model capacity and low-memory, low-latency serving.

References

[1] Pierre-Luc Bacon, Jean Harb, and Doina Precup. The option-critic architecture, 2016.

[2] Yogesh Balaji, Seungjun Nah, Xun Huang, Arash Vahdat, Jiaming Song, Qinsheng Zhang, Karsten Kreis, Miika Aittala, Timo Aila, Samuli Laine, Bryan Catanzaro, Tero Karras, and Ming-Yu Liu. ediff-i: Text-to-image diffusion models with an ensemble of expert denoisers, 2023.

[3] Weilin Cai, Juyong Jiang, Fan Wang, Jing Tang, Sunghun Kim, and Jiayi Huang. A survey on mixture of experts in large language models. *IEEE Transactions on Knowledge and Data Engineering*, page 1–20, 2025.

[4] Yekun Chai, Haoran Sun, Huang Fang, Shuohuan Wang, Yu Sun, and Hua Wu. Ma-rlhf: Reinforcement learning from human feedback with macro actions, 2025.

[5] Aidan Clark, Diego de Las Casas, Aurelia Guy, Arthur Mensch, Michela Paganini, Jordan Hoffmann, Bogdan Damoc, Blake Hechtman, Trevor Cai, Sebastian Borgeaud, et al. Unified scaling laws for routed language models. In *International conference on machine learning*, pages 4057–4086. PMLR, 2022.

[6] DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang,

We provide additional discussions of limitations and future directions in Section A4.

Han Bao, Hanwei Xu, Haocheng Wang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jiawei Wang, Jin Chen, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Litong Wang, Liyue Zhang, Meng Li, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qinyu Chen, Qiushi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runxin Xu, Ruoyu Zhang, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaoqing Wu, Shengfeng Ye, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Shuting Pan, T. Wang, Tao Yun, Tian Pei, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wanjia Zhao, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xiaokang Zhang, Xiaosha Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingkai Yu, Xinnan Song, Xinxia Shan, Xinyi Zhou, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Yang Zhang, Yanhong Xu, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Yu, Yi Zheng, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Ying Tang, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yu Wu, Yuan Ou, Yuchen Zhu, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yukun Zha, Yunfan Xiong, Yunxian Ma, Yuting Yan, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhipeng Xu, Zhiyu Wu, Zhongyu Zhang, Zhuoshu Li, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Ziyi Gao, and Zizheng Pan. Deepseek-v3 technical report, 2025.

[7] Xinhan Di and JoyJiaoW. Enhancing math reasoning in small-sized llms via preview difficulty-aware intervention, 2025.

[8] Artyom Eliseev and Denis Mazur. Fast inference of mixture-of-experts language models with offloading, 2023.

[9] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity, 2022.

[10] Zhida Feng, Zhenyu Zhang, Xintong Yu, Yewei Fang, Lanxin Li, Xuyi Chen, Yuxiang Lu, Jiaxiang Liu, Weichong Yin, Shikun Feng, Yu Sun, Li Chen, Hao Tian, Hua Wu, and Haifeng Wang. Ernie-vilg 2.0: Improving text-to-image diffusion model with knowledge-enhanced mixture-of-denoising-experts, 2023.

[11] Gemini-Team. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities, 2025.

[12] GLM-5-Team, :, Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin, Chendi Ge, Chenghua Huang, Chengxing Xie, Chenzheng Zhu, Congfeng Yin, Cunxiang Wang, Gengzheng Pan, Hao Zeng, Haoke Zhang, Haoran Wang, Huilong Chen, Jiajie Zhang, Jian Jiao, Jiaqi Guo, Jingsen Wang, Jingzhao Du, Jinzhu Wu, Kedong Wang, Lei Li, Lin Fan, Lucen Zhong, Mingdao Liu, Mingming Zhao, Pengfan Du, Qian Dong, Rui Lu, Shuang-Li, Shulin Cao, Song Liu, Ting Jiang, Xiaodong Chen, Xiaohan Zhang, Xuancheng Huang, Xuezhen Dong, Yabo Xu, Yao Wei, Yifan An, Yilin Niu, Yitong Zhu, Yuanhao Wen, Yukuo Cen, Yushi Bai, Zhongpei Qiao, Zihan Wang, Zikang Wang, Zilin Zhu, Ziqiang Liu, Zixuan Li, Bojie Wang, Bosi Wen, Can Huang, Changpeng Cai, Chao Yu, Chen Li, Chengwei Hu, Chenhui

Zhang, Dan Zhang, Daoyan Lin, Dayong Yang, Di Wang, Ding Ai, Erle Zhu, Fangzhou Yi, Feiyu Chen, Guohong Wen, Hailong Sun, Haisha Zhao, Haiyi Hu, Hanchen Zhang, Hanrui Liu, Hanyu Zhang, Hao Peng, Hao Tai, Haobo Zhang, He Liu, Hongwei Wang, Hongxi Yan, Hongyu Ge, Huan Liu, Huanpeng Chu, Jia'ni Zhao, Jiachen Wang, Jiajing Zhao, Jiamin Ren, Jiapeng Wang, Jiaxin Zhang, Jiayi Gui, Jiayue Zhao, Jijie Li, Jing An, Jing Li, Jingwei Yuan, Jinhua Du, Jinxin Liu, Junkai Zhi, Junwen Duan, Kaiyue Zhou, Kangjian Wei, Ke Wang, Keyun Luo, Laiqiang Zhang, Leigang Sha, Liang Xu, Lindong Wu, Lintao Ding, Lu Chen, Minghao Li, Nianyi Lin, Pan Ta, Qiang Zou, Rongjun Song, Ruiqi Yang, Shangqing Tu, Shangtong Yang, Shaoxiang Wu, Shengyan Zhang, Shijie Li, Shuang Li, Shuyi Fan, Wei Qin, Wei Tian, Weining Zhang, Wenbo Yu, Wenjie Liang, Xiang Kuang, Xiangmeng Cheng, Xiangyang Li, Xiaoquan Yan, Xiaowei Hu, Xiaoying Ling, Xing Fan, Xingye Xia, Xinyuan Zhang, Xinze Zhang, Xirui Pan, Xu Zou, Xunkai Zhang, Yadi Liu, Yandong Wu, Yanfu Li, Yidong Wang, Yifan Zhu, Yijun Tan, Yilin Zhou, Yiming Pan, Ying Zhang, Yinpei Su, Yipeng Geng, Yong Yan, Yonglin Tan, Yuean Bi, Yuhan Shen, Yuhao Yang, Yujiang Li, Yunan Liu, Yunqing Wang, Yuntao Li, Yurong Wu, Yutao Zhang, Yuxi Duan, Yuxuan Zhang, Zezhen Liu, Zhengtao Jiang, Zhenhe Yan, Zheyu Zhang, Zhixiang Wei, Zhuo Chen, Zhuoer Feng, Zijun Yao, Ziwei Chai, Ziyuan Wang, Zuzhou Zhang, Bin Xu, Minlie Huang, Hongning Wang, Juanzi Li, Yuxiao Dong, and Jie Tang. Glm-5: from vibe coding to agentic engineering, 2026.

[13] Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. Minillm: On-policy distillation of large language models, 2026.

[14] Jean Harb, Pierre-Luc Bacon, Martin Klissarov, and Doina Precup. When waiting is not an option : Learning options with a deliberation cost, 2017.

[15] Xu Owen He. Mixture of a million experts. *arXiv preprint arXiv:2407.04153*, 2024.

[16] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding, 2021.

[17] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset, 2021.

[18] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021.

[19] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Lélio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Théophile Gervet, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mixtral of experts, 2024.

[20] Martin Klissarov and Doina Precup. Flexible option learning, 2021.

[21] Seijin Kobayashi, Yanick Schimpf, Maximilian Schlegel, Angelika Steger, Maciej Wolczyk, Johannes von Oswald, Nino Scherrer, Kaitlin Maile, Guillaume Lajoie, Blake A. Richards, Rif A. Saurous, James Manyika, Blaise Agüera y Arcas, Alexander Meulemans, and João Sacramento. Emergent temporal abstractions in autoregressive models enable hierarchical reinforcement learning, 2025.

[22] Enshu Liu, Junyi Zhu, Zinan Lin, Xuefei Ning, Matthew B. Blaschko, Shengen Yan, Guohao Dai, Huazhong Yang, and Yu Wang. Efficient expert pruning for sparse mixture-of-experts language models: Enhancing performance and reducing inference costs, 2024.

[23] Kevin Lu. On-policy distillation, 2025.

[24] Xudong Lu, Qi Liu, Yuhui Xu, Aojun Zhou, Siyuan Huang, Bo Zhang, Junchi Yan, and Hongsheng Li. Not all experts are equal: Efficient expert pruning and skipping for mixture-of-experts large language models, 2024.

[25] Alexandre Muzio, Alex Sun, and Churan He. Seer-moe: Sparse expert efficiency through regularization for mixture-of-experts, 2024.

[26] Dhruv Nathawani, Shuoyang Ding, Vitaly Lavrukhin, Igor Gitman, Somshubra Majumdar, Evelina Bakhturina, Boris Ginsburg, and Jane Polak Scowcroft. Nemotron-Post-Training-Dataset-v2, August 2025.

[27] OpenAI, :, Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K. Arora, Yu Bai, Bowen Baker, Haiming Bao, Boaz Barak, Ally Bennett, Tyler Bertao, Nivedita Brett, Eugene Brevdo, Greg Brockman, Sebastien Bubeck, Che Chang, Kai Chen, Mark Chen, Enoch Cheung, Aidan Clark, Dan Cook, Marat Dukhan, Casey Dvorak, Kevin Fives, Vlad Fomenko, Timur Garipov, Kristian Georgiev, Mia Glaese, Tarun Gogineni, Adam Goucher, Lukas Gross, Katia Gil Guzman, John Hallman, Jackie Hehir, Johannes Heidecke, Alec Helyar, Haitang Hu, Romain Huet, Jacob Huh, Saachi Jain, Zach Johnson, Chris Koch, Irina Kofman, Dominik Kundel, Jason Kwon, Volodymyr Kyrylov, Elaine Ya Le, Guillaume Leclerc, James Park Lennon, Scott Lessans, Mario Lezcano-Casado, Yuanzhi Li, Zhuohan Li, Ji Lin, Jordan Liss, Lily, Liu, Jiancheng Liu, Kevin Lu, Chris Lu, Zoran Martinovic, Lindsay McCallum, Josh McGrath, Scott McKinney, Aidan McLaughlin, Song Mei, Steve Mostovoy, Tong Mu, Gideon Myles, Alexander Neitz, Alex Nichol, Jakub Pachocki, Alex Paino, Dana Palmie, Ashley Pantuliano, Giambattista Parascandolo, Jongsoo Park, Leher Pathak, Carolina Paz, Ludovic Peran, Dmitry Pimenov, Michelle Pokrass, Elizabeth Proehl, Huida Qiu, Gaby Raila, Filippo Raso, Hongyu Ren, Kimmy Richardson, David Robinson, Bob Rotsted, Hadi Salman, Suvansh Sanjeev, Max Schwarzer, D. Sculley, Harshit Sikchi, Kendal Simon, Karan Singhal, Yang Song, Dane Stuckey, Zhiqing Sun, Philippe Tillet, Sam Toizer, Foivos Tsimpourlas, Nikhil Vyas, Eric Wallace, Xin Wang, Miles Wang, Olivia Watkins, Kevin Weil, Amy Wendling, Kevin Whinnery, Cedric Whitney, Hannah Wong, Lin Yang, Yu Yang, Michihiro Yasunaga, Kristen Ying, Wojciech Zaremba, Wenting Zhan, Cyril Zhang, Brian Zhang, Eddie Zhang, and Shengjia Zhao. gpt-oss-120b and gpt-oss-20b model card, 2025.

[28] Dustin Podell, Zion English, Kyle Lacey, Andreas Blattmann, Tim Dockhorn, Jonas Müller, Joe Penna, and Robin Rombach. Sdxl: Improving latent diffusion models for high-resolution image synthesis, 2023.

[29] Qwen Team. Qwen3.5: Towards native multimodal agents, February 2026.

[30] QwenTeam. Qwen3-next: Towards ultimate training and inference efficiency, 2025.

[31] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation, 2018.

[32] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer, 2017.

[33] Xiaoniu Song, Zihang Zhong, Rong Chen, and Haibo Chen. Promoe: Fast moe-based llm serving using proactive caching, 2025.

[34] Mingjie Sun, Zhuang Liu, Anna Bair, and J. Zico Kolter. A simple and effective pruning approach for large language models, 2024.

[35] Richard S. Sutton, Doina Precup, and Satinder Singh. Between mdps and semi-mdps: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1):181–211, 1999.

[36] Suraiya Tairin, Shohaib Mahmud, Haiying Shen, and Anand Iyer. emoe: Task-aware memory efficient mixture-of-experts-based (moe) model inference, 2025.

[37] Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. TRL: Transformers Reinforcement Learning, 2020.

[38] Team Wan, Ang Wang, Baole Ai, Bin Wen, Chaojie Mao, Chen-Wei Xie, Di Chen, Feiwu Yu, Haiming Zhao, Jianxiao Yang, Jianyuan Zeng, Jiayu Wang, Jingfeng Zhang, Jingren Zhou, Jinkai Wang, Jixuan Chen, Kai Zhu, Kang Zhao, Keyu Yan, Lianghua Huang, Mengyang Feng, Ningyi Zhang, Pandeng Li, Pingyu Wu, Ruihang Chu, Ruili Feng, Shiwei Zhang, Siyang Sun, Tao Fang, Tianxing Wang, Tianyi Gui, Tingyu Weng, Tong Shen, Wei Lin, Wei Wang, Wei Wang, Wenmeng Zhou, Wente Wang, Wenting Shen, Wenyuan Yu, Xianzhong Shi, Xiaoming Huang, Xin Xu, Yan Kou, Yangyu Lv, Yifei Li, Yijing Liu, Yiming Wang, Yingya Zhang, Yitong Huang, Yong Li, You Wu, Yu Liu, Yulin Pan, Yun Zheng, Yuntao Hong, Yupeng Shi, Yutong Feng, Zeyinzi Jiang, Zhen Han, Zhi-Fan Wu, and Ziyu Liu. Wan: Open and advanced large-scale video generative models, 2025.

[39] Yanyue Xie, Zhi Zhang, Ding Zhou, Cong Xie, Ziang Song, Xin Liu, Yanzhi Wang, Xue Lin, and An Xu. Moe-pruner: Pruning mixture-of-experts large language model using the hints from its router, 2024.

[40] Leyang Xue, Yao Fu, Zhan Lu, Luo Mai, and Mahesh Marina. Moe-infinity: Efficient moe inference on personal machines with sparsity-aware expert cache, 2025.

[41] Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Ruslan Salakhutdinov, and Alexander Smola. Deep sets, 2018.

[42] Yuning Zhang, Grant Pinkert, Nan Yang, Yanli Li, and Dong Yuan. Duoserve-moe: Dual-phase expert prefetch and cache scheduling for efficient moe llm inference, 2025.

A1. Related Works

Mixture-of-Experts Models. MoE architectures have become the dominant paradigm for scaling the capacity for LLMs. Recent advancements in MoE architectures have shifted towards a high-sparsity regime, where the total number of experts far exceeds the number of active experts per layer. For instance, [27] has 128 experts per layer but only activates 4 per token; [30] has 512 experts per layer, but only activates 10 per token (plus one shared expert).

MoE architectures are also becoming increasingly popular among diffusion models. Many diffusion models, from text-to-image models such as SDXL [28], ERNIE-ViLG 2.0 [10], and eDIFF-I [2], to recent video generation models such as Wan2.2 [38], use different denoisers (as experts) for different denoising stages.

MoE Efficiency. Several works have aimed to improve MoE efficiency via pruning, caching / prefetching, and offloading-aware serving.

On the pruning side, [25] proposes a two-stage method that prunes the total number of experts using frequency count and then applies fine-tuning to recover accuracy. [24] minimizes a reconstruction loss and choose the subset of experts that best reconstructs the original layer's output and shows that this beats dropping least-used experts. [22] introduces Efficient Expert Pruning (EEP) that employs gradient-free evolutionary strategy to prune and merge experts. [39] designs a novel pruning metric that incorporates MoE router weights information to identify and remove unimportant weights in expert layers.

On the caching / prefetching and offloading-aware serving side, [40] presents MoE-infinity that offloads certain experts to host memory and allows memory-constrained GPUs to serve MoE models. It uses expert activations to predict the set of experts to cache and prefetch. [33] studies a similar setting and proposes ProMoE, which also uses activations to predict which experts will be needed soon and prefetch them. [36] demonstrates that the set of experts that are needed across different layers and between similar prompts are correlated, and presents eMoE that predicts the set of experts to load onto GPU based on these observations in a task-aware manner. [42] introduces DuoServe-MoE, which is an inference serving system that offloads certain expert weights to CPU and uses different scheduling for prefilling and decoding. Works in this thread generally do not study the tradeoff between latency cost and generation quality, but shares the similar goal of predicting the set of experts that will be used.

Options, s-MDPs, and hierarchical RL. We cast expert-mask selection as a temporally extended control problem, where expert masks are options, switching the expert mask corresponds to option termination and can be regularized via an explicit switching latency cost. [35] formalizes the options framework, showing that adding options to MDP induces a semi-MDP (s-MDP). [1] derives policy-gradient theorems for options and proposes the option-critic architecture for options learning. [14] points out that the options framework is the most useful when there's a deliberation cost between different options, where temporally extended structure naturally arises, and presents the variant of option-critic with deliberation cost. [20] revisits intra-option learning in the context of deep reinforcement learning in order to enable updating all options consistent with current primitive action choices, leading to better performance and data efficiency in option discovery.

Several recent papers are also utilizing the options framework or hierarchical RL to train language models. For example, [4] frames sequence of tokens as macro-actions and incorporates them into RLHF. [7] proposes a hierarchical RL method based on GRPO for reasoning. [21] proposes "Internal RL," a hierarchical RL framework that improves sample efficiency in sparse-reward tasks by discovering and steering the temporally abstract action representations that naturally emerge within residual streams of pretrained autoregressive models.

A2. Additional Implementation Details

Full algorithm. We present the full training procedure in Algorithm 2.

Algorithm 2: MoE Option-Critic Training (Full)

Input: MoE model with L layers, N experts per layer, top-$\tilde{k}$ routing; prompt dataset $\mathcal{D}$; teacher model p_{teacher}; teacher mixing coefficient τ; discount γ; GAE parameter λ; deliberation cost η, learning rates $\alpha_{\text{controller}}$, α_{intra}. We use θ to denote the LLM parameters, ν to denote the termination head, ψ to denote the critic parameters (including V_Ω and Q_Ω), and ϕ to denote the option selection head parameters.

for *each training iteration* **do**

- Sample prompt $x \sim \mathcal{D}$;
- `// Rollout with teacher mixing`
- $\omega_0^{(\ell)} \leftarrow \mathrm{TopK}\big(g_0^{(\ell)}, \hat{k}\big)$ for each layer ℓ, where $g_0^{(\ell)} \in \mathbb{R}^N$ are the router logits at $t=0$;
- **for** $t = 1, \ldots, T$ **do**
  - **for** *each layer* ℓ **do**
    - $\beta_t^{(\ell)} \leftarrow \sigma\left(\mathrm{MLP}_\beta\left(\mathrm{concat}\left(\bar{h}_t^{(\ell)}, \bar{z}^{(\ell)}(\omega_{t-1}^{(\ell)})\right)\right)\right)$;
    - Sample $d_t^{(\ell)} \sim \mathrm{Bern}(\beta_t^{(\ell)})$;
    - $\omega_t^{(\ell)} \leftarrow \begin{cases} \text{PL-sample}(f_{\text{sel}}(h_t^{(\ell)}), \hat{k}) & \text{if } d_t^{(\ell)}=1 \\ \omega_{t-1}^{(\ell)} & \text{otherwise} \end{cases}$;
    - Mask router to experts in $\omega_t^{(\ell)}$;
  - $p_{\text{mix}} \leftarrow (1-\tau)\,\pi_{\omega,\theta}(\cdot \mid x, a_{<t}) + \tau\, p_{\text{teacher}}(\cdot \mid x, a_{<t})$;
  - Sample $a_t \sim p_{\text{mix}}$;
  - $w_t \leftarrow \pi_{\omega,\theta}(a_t)/p_{\text{mix}}(a_t)$;
  - $r_t \leftarrow \log p_{\text{teacher}}(a_t \mid x, a_{<t}) - \log \pi_{\omega,\theta}(a_t \mid x, a_{<t})$;

Controller architecture details. Each MoE layer has an independent controller. The DeepSets expert set encoder uses a learned embedding dimension of $d_e = 128$ and a two-layer MLP with GELU activation and hidden dimension 1024. The termination head, option-value head Q_Ω, and expert selection head all use hidden dimension 1024. The state-value head V_Ω is a single linear layer initialized from the router weights. The termination head's bias is initialized to -3 (corresponding to an initial switch probability of $\sigma(-3) \approx 0.05$, encouraging temporal continuity from the beginning). RMSNorm is applied to balance the scale of $h_t^{(\ell)}$ and $z_t^{(\ell)}$ before concatenation.

Advantage normalization For the termination gradient, the raw advantage is

$$A_t^{\text{term},(\ell)} = Q_\Omega(h_t^{(\ell)}, \omega_{t-1}^{(\ell)}) - V_\Omega(h_t^{(\ell)}) + \eta.$$

We apply RMS normalization (without mean centering) independently within each layer ℓ over all timesteps $t > 0$:

$$\hat{A}_t^{\text{term},(\ell)} = \frac{A_t^{\text{term},(\ell)}}{\mathrm{RMS}(A^{\text{term},(\ell)})}, \quad \text{where} \quad \mathrm{RMS}(A^{\text{term},(\ell)}) = \sqrt{\frac{1}{T-1}\sum_{t=1}^{T-1}\left(A_t^{\text{term},(\ell)}\right)^2}.$$

Algorithm 2 continued.;
for *same training iteration* **do**
  `// Per-layer critic and termination gradient computation`
  for *each layer* ℓ **do**
    for $t = T, \ldots, 1$ **do** `// GAE(`λ`)`
      $\beta_{t+1}^{(\ell)} \leftarrow \sigma\left(\mathrm{MLP}_\beta\left(\mathrm{concat}\left(\bar{h}_{t+1}^{(\ell)}, \bar{z}^{(\ell)}(\omega_t^{(\ell)})\right)\right)\right)$;
      $U_t \leftarrow \beta_{t+1}^{(\ell)} V_\Omega(h_{t+1}^{(\ell)}) + (1 - \beta_{t+1}^{(\ell)}) Q_\Omega(h_{t+1}^{(\ell)}, \omega_t^{(\ell)})$;
      $\delta_t^V \leftarrow r_t + \gamma V_\Omega(h_{t+1}^{(\ell)}) - V_\Omega(h_t^{(\ell)})$;
      $\hat{A}_t^V \leftarrow \delta_t^V + \gamma\lambda \hat{A}_{t+1}^V$;
      $\delta_t^Q \leftarrow r_t + \gamma U_t - Q_\Omega(h_t^{(\ell)}, \omega_t^{(\ell)})$;
      $\hat{A}_t^Q \leftarrow \delta_t^Q + \gamma\lambda \hat{A}_{t+1}^Q$;
    for $t = 1, \ldots, T$ **do**
      $\hat{V}_t^{\mathrm{targ}} \leftarrow V_\Omega(h_t^{(\ell)}) + \hat{A}_t^V$; $\hat{Q}_t^{\mathrm{targ}} \leftarrow Q_\Omega(h_t^{(\ell)}, \omega_t^{(\ell)}) + \hat{A}_t^Q$;
      $d\nu \mathrel{-}= w_t \cdot \nabla_\nu \beta(h_t^{(\ell)}, \omega_{t-1}^{(\ell)}) \cdot (Q_\Omega(h_t^{(\ell)}, \omega_{t-1}^{(\ell)}) - V_\Omega(h_t^{(\ell)}) + \eta)$;
      if $d_t^{(\ell)} = 1$ **then**
        $d\phi \mathrel{+}= w_t \cdot \nabla_\phi \log \pi_{\mathrm{sel}}(\omega_t^{(\ell)} \mid h_t^{(\ell)}) \cdot (Q_\Omega(h_t^{(\ell)}, \omega_t^{(\ell)}) - V_\Omega(h_t^{(\ell)}))$;
      $d\psi \mathrel{-}= \nabla_\psi\left[\left(V_\Omega(h_t^{(\ell)}) - \hat{V}_t^{\mathrm{targ}}\right)^2 + \left(Q_\Omega(h_t^{(\ell)}, \omega_t^{(\ell)}) - \hat{Q}_t^{\mathrm{targ}}\right)^2\right]$;
  `// Intra-option policy gradient computation`
  for $t = 1, \ldots, T$ **do**
    $\bar{G}_t \leftarrow \sum_{j \geq 0} \gamma^j r_{t+j}$;
    $d\theta \mathrel{+}= w_t \cdot \nabla_\theta \log \pi_{\omega,\theta}(a_t \mid x, a_{<t}) \cdot \bar{G}_t$;
  $(\nu, \psi, \phi) \leftarrow (\nu, \psi, \phi) + \alpha_{\mathrm{controller}} \cdot \frac{1}{L}(d\nu, d\psi, d\phi)$; $\theta \leftarrow \theta + \alpha_{\mathrm{intra}} \cdot d\theta$;

This preserves the sign of advantages while stabilizing scale, which we found to be important for the stability of training.

When a switch occurs ($d_t^{(\ell)} = 1$), the per-layer option selection advantage is

$$A_t^{\mathrm{sel},(\ell)} = Q_\Omega(h_t^{(\ell)}, \omega_t^{(\ell)}) - V_\Omega(h_t^{(\ell)}).$$

We apply the same RMS normalization, computed over switch positions only:

$$\hat{A}_t^{\mathrm{sel},(\ell)} = \frac{A_t^{\mathrm{sel},(\ell)}}{\mathrm{RMS}(A^{\mathrm{sel},(\ell)})}, \quad \text{where} \quad \mathrm{RMS}(A^{\mathrm{sel},(\ell)}) = \sqrt{\frac{1}{|\mathcal{S}^{(\ell)}|} \sum_{t \in \mathcal{S}^{(\ell)}} (A_t^{\mathrm{sel},(\ell)})^2},$$

and $\mathcal{S}^{(\ell)} = \{t : d_t^{(\ell)} = 1, t > 0\}$ is the set of switch positions at layer ℓ.

The intra-option policy gradient (Theorem 1) requires $Q_U(s, \omega, a)$, the value of taking action a under state-option pair (s, ω). We estimate this with the Monte Carlo return:

$$\bar{G}_t = \sum_{j=0}^{T-t-1} \gamma^j r_{t+j},$$

which serves as an unbiased estimate of $Q_U(s_t, \omega_t, a_t)$. We use $\bar{G}_t$ directly as the advantage (without subtracting a baseline as in [14], which we found to work better empirically). Before applying the policy gradient, we standardize $\bar{G}_t$ across all T response tokens:

$$\hat{A}_t^{\text{intra}} = \frac{\bar{G}_t - \mu}{\sigma}, \quad \text{where} \quad \mu = \frac{1}{T}\sum_{t=1}^{T}\bar{G}_t, \quad \sigma = \sqrt{\frac{1}{T}\sum_{t=1}^{T}(\bar{G}_t - \mu)^2}.$$

This advantage is shared across all layers.

Implementation detail of reconstruction loss minimization. For reconstruction loss minimization, [24] prunes out a set of experts so that the remaining set of experts minimizes a reconstruction loss. For each MoE layer ℓ, we cache input-output pairs $(x_i, \mathcal{F}(x_i))$ by running a forward pass over the calibration data, where $\mathcal{F}$ denotes the full MoE layer. Given a candidate subset $\mathbf{C} \subseteq \{1, \ldots, N\}$ with $|\mathbf{C}| = \hat{k}$, the pruned layer $\mathcal{F}'(\cdot, \mathbf{C})$ masks all experts not in $\mathbf{C}$, recomputes the top-$\tilde{k}$ routing and softmax normalization over only the allowed experts, and produces a weighted combination of expert outputs. The objective is

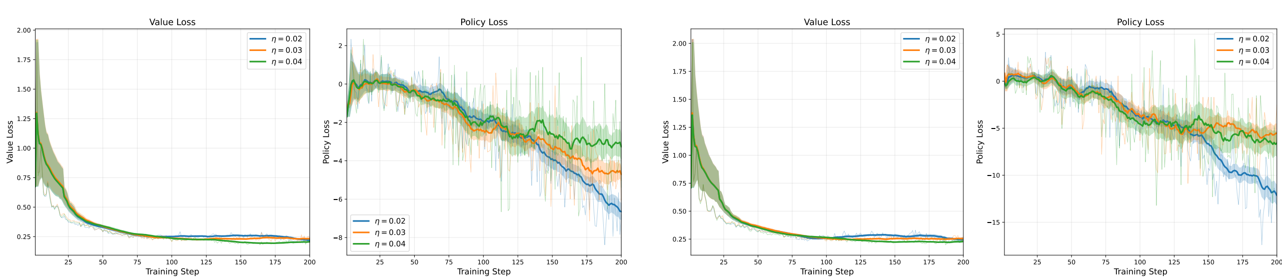
$$\min_{\mathbf{C}:|\mathbf{C}|=\hat{k}} \left\| \mathcal{F}'(X, \mathbf{C}) - \mathcal{F}(X) \right\|_F ,$$

where X denotes the cached calibration inputs and $\|\cdot\|_F$ is the Frobenius norm. The original method in [24] solves this via exhaustive enumeration over all $\binom{N}{\hat{k}}$ subsets, which is feasible in their setting where $N = 8$. In our setting with $N = 32$ experts, exhaustive search is computationally infeasible. We therefore employ a greedy forward selection procedure: starting from an empty set, we iteratively add the expert that yields the largest reduction in reconstruction loss, repeating $\hat{k}$ times. This compromise is due to the inherent limitation of the scalability of their method, and our implementation is our best effort to map their method in our setting.

A3. Additional Experimental Results

In this section, we present additional experimental results and analysis.

Loss curves. We present the loss curves in Figure A1. We can see that the value loss steadily decreases, which indicates that the value and option-value heads are learning the value of states and state-option pairs effectively.

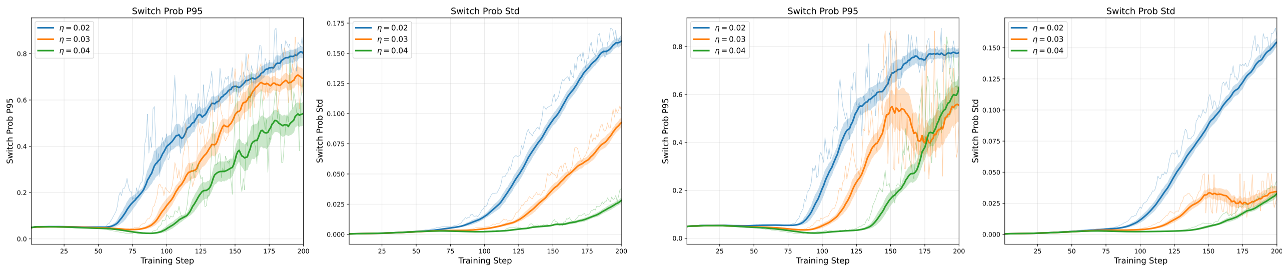


(a) Loss curves for $\hat{k} = 8$.

(b) Loss curves for $\hat{k} = 16$

Fig. A1: Loss curves for $\hat{k} = 8$ and $\hat{k} = 16$, respectively. Running averages and confidence bands plotted the same way as in Figure 4.

Additional switch rate curves. We present additional switch rate curves in Figure A2. For each subplot, the plot on the left presents the switch probability at 95 percentile over the trajectory throughout training, while the plot on the right presents the standard deviation of the switch probabilities over the trajectory throughout training. We can see that though switch rate first decreases and then slightly increases (as shown in Figure 4), both the 95 percentile switch rate and the standard deviation of the switch rates steadily increases, indicating that the termination head is learning to distinguish when to switch and when to not switch effectively.

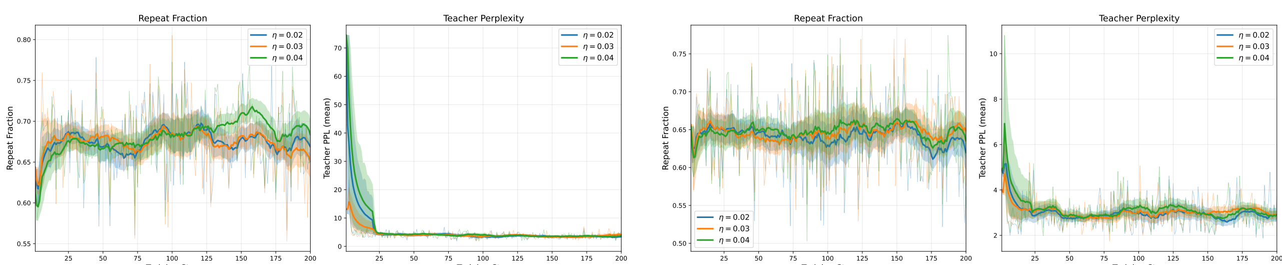


(a) Switch rate curves for $\hat{k} = 8$.

(b) Switch rate curves for $\hat{k} = 16$.

Fig. A2: Additional switch rate curves for $\hat{k} = 8$ and $\hat{k} = 16$, respectively. Running averages and confidence bands plotted the same way as in Figure 4.

Training stability. A common failure mode of MoE models under constrained routing is catastrophic repetition, where the model degenerates into producing repetitive content (concrete examples in Section A5). Figure A3 shows that our method avoids this failure mode: the repetition rate ($1 -$ fraction of unique tokens per trajectory) remains stable within a healthy range throughout training for both $\hat{k} = 8$ and $\hat{k} = 16$. The right panels show the perplexity of the frozen teacher model (i.e., the original `gpt-oss-20b` without any controllers or weight updates) evaluated on the student's generated trajectories, which decreases over training, indicating that the student's outputs become more aligned with the teacher rather than diverging.

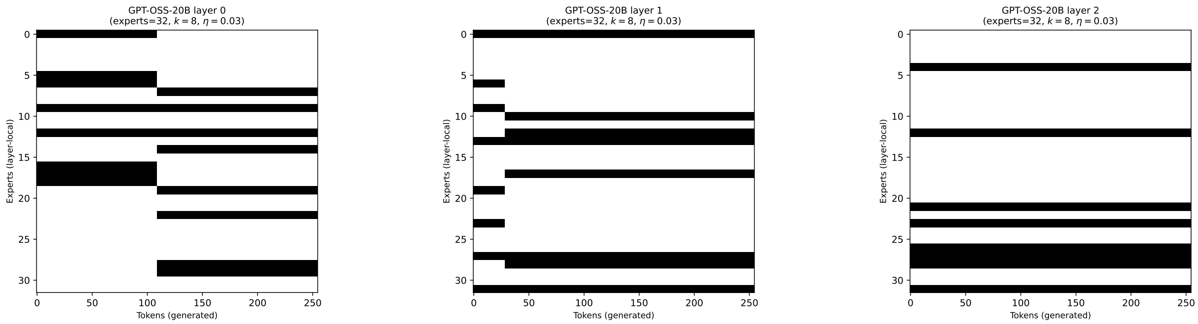


(a) Repetition rate and perplexity curves for $\hat{k} = 8$.

(b) Repetition rate and perplexity curves for $\hat{k} = 16$.

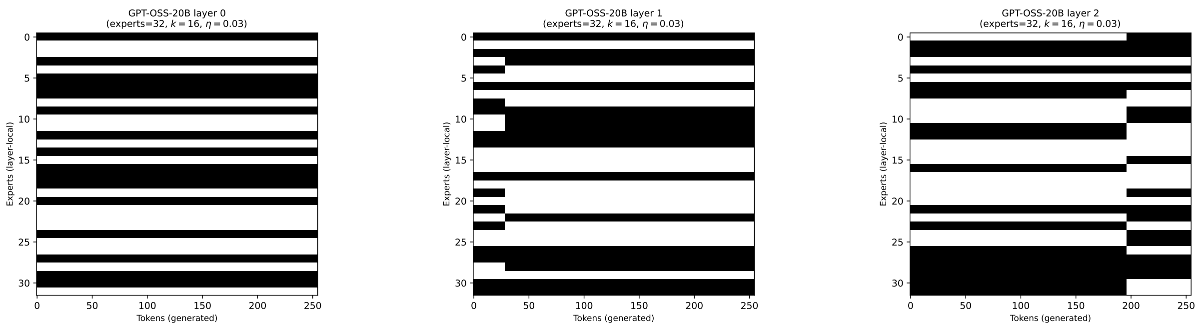
Fig. A3: Repetition rate and perplexity curves for $\hat{k} = 8$ and $\hat{k} = 16$, respectively. Running averages and confidence bands plotted the same way as in Figure 4.

Temporal continuity with controller. We present additional plots demonstrating temporal continuity of options under our trained controller using the same setup as in Section 5. Figure A4 and A5 plot the active options throughout the trajectory for $\eta = 0.03$, while Figure A6 and Figure A7 plot for $\eta = 0.04$. We can see that options almost always demonstrate significant temporal continuity across different layers and different deliberation costs.



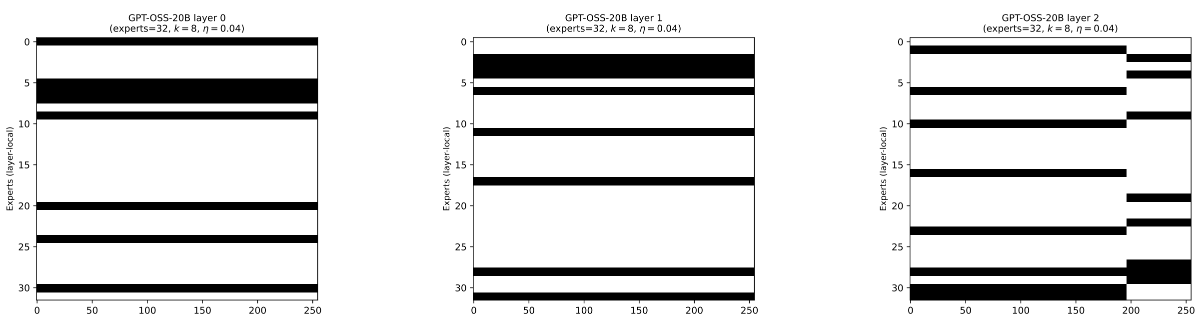
(a) Layer 0 (b) Layer 1 (c) Layer 2

Fig. A4: Options in layer 0, 1, 2 throughout the trajectory with gpt-oss-20b, $\hat{k} = 8, \eta = 0.03$.



(a) Layer 0 (b) Layer 1 (c) Layer 2

Fig. A5: Options in layer 0, 1, 2 throughout the trajectory with gpt-oss-20b, $\hat{k} = 16, \eta = 0.03$.



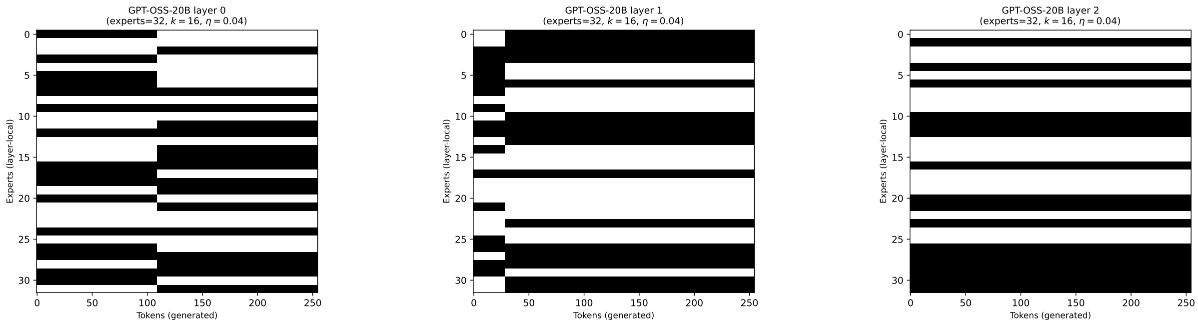
(a) Layer 0 (b) Layer 1 (c) Layer 2

Fig. A6: Options in layer 0, 1, 2 throughout the trajectory with gpt-oss-20b, $\hat{k} = 8, \eta = 0.04$.

A4. Limitations and Future Directions

Our work proposes temporally extended MoE as a design philosophy and demonstrates its feasibility through a principled option-critic framework. We discuss several limitations and promising directions for future research.

From philosophy to deployment. While we identify three concrete opportunities enabled by temporal extension — memory-efficient inference, chunk-wise training, and continual learning with expandable experts



(a) Layer 0 (b) Layer 1 (c) Layer 2

Fig. A7: Options in layer 0, 1, 2 throughout the trajectory with gpt-oss-20b, $\hat{k} = 16, \eta = 0.04$.

— our current experiments focus on validating that temporally extended routing can be learned with graceful degradation in generation quality. Building end-to-end systems that realize these memory savings is substantial systems work that we leave to future research. Similarly, the deliberation cost in our framework is a tunable hyperparameter rather than a measured hardware latency; grounding it in actual expert loading times for a specific deployment target would make the cost-quality tradeoff directly actionable.

Realizing temporal continuity in pre-training. Our framework does post-training on a pre-trained MoE model. An intriguing alternative is to encode temporal continuity directly into the pre-training objective. As GPU memory becomes an increasingly binding constraint for scaling, building temporal structure into routing from the start could yield models that are inherently more memory-friendly without requiring a separate controller. Moreover, natural language itself exhibits rich temporal structure (e.g., topics, arguments, or reasoning chains that persist over extended spans within a trajectory) and there may be richer connections between temporally abstract representations in language and temporally extended expert routing that future work can exploit.

Per-layer vs. cross-layer options. In our framework, each layer maintains its own independent option, meaning that expert masks may switch at different token positions across layers. In practice, the most straightforward way to realize the memory benefits discussed in Section 3 would be to switch expert masks across all layers simultaneously, so that a single offload/onload event can swap the entire active expert set at once. This would also simplify chunk-wise training, where a chunk boundary aligned across all layers naturally defines the segments for memory-efficient forward-backward passes. However, defining options as joint expert masks across all L layers would result in a combinatorially larger option space, making the learning problem significantly harder. Our per-layer formulation is a practical compromise that makes learning tractable while still demonstrating the benefits of temporal extension. Bridging this gap is an important direction for future work.

Evaluation scope. Our evaluation covers three benchmarks that span distinct capabilities: mathematical reasoning (MATH), broad knowledge (MMLU), and multilingual understanding (MMMLU). However, we have not yet included other important dimensions such as code generation, long-form instruction following, or open-ended conversation. In addition, we evaluate on 200 randomly selected questions per benchmark rather than the full test sets, which is sufficient to establish statistically significant comparisons (as reflected in the confidence intervals in Tables 2 and 3) but may not fully capture performance variation across fine-grained subtopics. A more comprehensive evaluation across a wider range of benchmarks and larger sample sizes would strengthen the empirical picture.

Disentangling temporal extension from self-distillation. Our method combines two sources of improvement over static pruning baselines: (1) the ability to dynamically switch expert masks across tokens, and (2) on-policy self-distillation that adapts the model's weights to better match the base model. Note that the latter is not an ad hoc addition, but an inherent component of the option-critic framework. Drawing this connection between between intra-option learning in option-critic framework and on-policy distillation is an important part of our contribution. Nonetheless, since our primary baselines [24] are training-free post-training methods, one might ask whether the gains we observe stem primarily from temporal extension or from the additional parameter updates. To fully disentangle these two factors, a natural ablation would be to apply the same on-policy distillation objective to a statically pruned model with a fixed expert mask, isolating the effect of self-distillation alone. We leave this ablation to future work and note that it would provide a more fine-grained understanding of the relative contributions of dynamic routing and weight adaptation.

A5. Examples

In this section, we present some concrete rollouts of different methods compared in the paper. We look at a number theory problem in MATH dataset, using $\hat{k} = 16$ experts for each method. We set token sampling temperature to 0.5 and max token count to 2048. For our method, we use a deliberation cost of 0.02 and take the checkpoint at 120$^{\text{th}}$ step. While all methods are capable of producing coherent outputs to different extents, this particular example illustrates a case where only our method maintains coherent reasoning, whereas the baseline methods degrade to varying degrees of gibberish.

Prompt: Problem: The least common multiple of $1! + 2!$, $2! + 3!$, $3! + 4!$, $4! + 5!$, $5! + 6!$, $6! + 7!$, $7! + 8!$, and $8! + 9!$ can be expressed in the form $a \cdot b!$, where a and b are integers and b is as large as possible. What is $a + b$?

Response: (Ours) We need to find the LCM of numbers: $n_i = i! + (i+1)!$ for i from 1 to 8 inclusive.

Compute each expression: $i! + (i+1)! = i! + (i+1) \cdot i! = (1 + (i+1))i! = (i+2)i!$.

So $n_i = (i+2) \cdot i!$ for $i = 1 \ldots 8$.

Thus $n_i = (i+2) \cdot i!$.

We need the LCM of these numbers.

We can express each as $(i+2) \cdot i!$. Let's find prime factorizations.

But maybe we can compute the LCM using factorials.

Observe that $i!$ divides $(i+1)!$, etc. But we have an $(i+2)$ factor.

We need the LCM of numbers of the form $(i+2) \cdot i!$.

We can factor each as $(i+2) \cdot i!$.

We can compute the LCM by considering prime powers.

Let's compute for each i:

$i = 1$: $1! + 2! = 1 + 2 = 3$. Our formula: $(1 + 2) \cdot 1! = 3 \cdot 1 = 3$.

$i = 2$: $2! + 3! = 2 + 6 = 8$. Formula: $(2 + 2) \cdot 2! = 4 \cdot 2 = 8$.

$i = 3$: $3! + 4! = 6 + 24 = 30$. Formula: $(3 + 2) \cdot 3! = 5 \cdot 6 = 30$.

$i = 4$: $4! + 5! = 24 + 120 = 144$. Formula: $(4 + 2) \cdot 4! = 6 \cdot 24 = 144$.

$i = 5$: $5! + 6! = 120 + 720 = 840$. Formula: $(5 + 2) \cdot 5! = 7 \cdot 120 = 840$.

$i = 6$: $6! + 7! = 720 + 5040 = 5760$. Formula: $(6 + 2) \cdot 6! = 8 \cdot 720 = 5760$.

$i = 7$: $7! + 8! = 5040 + 40320 = 45360$. Formula: $(7 + 2) \cdot 7! = 9 \cdot 5040 = 45360$.

$i = 8$: $8! + 9! = 40320 + 362880 = 403200$. Formula: $(8 + 2) \cdot 8! = 10 \cdot 40320 = 403200$.

Thus the numbers are: $3, 8, 30, 144, 840, 5760, 45360, 403200$.

We need the LCM of all these numbers.

We can compute the LCM by factoring each.

Let's factor each:

3: prime 3.

8: 2^3.

30: $2 \cdot 3 \cdot 5$.

144: $2^4 \cdot 3^2$.

840: $2^3 \cdot 5 \cdot 7$.

5760: $2^6 \cdot 3^2 \cdot 5$.

45360: $2^4 \cdot 3^4 \cdot 5 \cdot 7$? Let's compute: $45360/16 = 2835$; $2835/3 = 945$; $945/3 = 315$; $315/3 = 105$; $105/5 = 21$; $21/7 = 3$; $3/3 = 1$. So prime factors: $2^4 \cdot 3^4 \cdot 5 \cdot 7$.

403200: Let's factor: $403200/10 = 40320$; $40320/2 = 20160$; $20160/2 = 10080$; $10080/2 = 5040$; $5040/2 = 2520$; $2520/2 = 1260$; $1260/2 = 630$; $630/2 = 315$; $315/3 = 105$; $105/3 = 35$; $35/5 = 7$; $7/7 = 1$. So prime factors: $2^7 \cdot 3^2 \cdot 5^1 \cdot 7^1$? Wait we counted: $403200 = 10 \cdot 40320 = 2 \cdot 5 \cdot 40320$; $40320 = 2^7 \cdot 3^2 \cdot 5$? Let's compute again: $40320 = 2^7 \cdot 3^2 \cdot 5$? Actually $40320 = 8! = 40320$. $8! = 2^7 \cdot 3^2 \cdot 5^1$? Let's check: $8! = 40320$. Factor: $40320 = 2^7 \cdot 3^2 \cdot 5^1$? Let's compute: $2^7 = 128$; $3^2 = 9$; $5 = 5$; $128 \cdot 9 = 1152$; $1152 \cdot 5 = 5760$; 5760? Wait 5760 is 6!; $5760 \cdot 7$? Actually $40320 = 5760 \cdot 7$? $5760 \cdot 7 = 40320$. So $40320 = 2^6 \cdot 3^2 \cdot 5 \cdot 7$? Let's check: $5760 = 2^6 \cdot 3^2 \cdot 5$; times 7 gives 40320. So $40320 = 2^6 \cdot 3^2 \cdot 5 \cdot 7$. Then $403200 = 10 \cdot 40320 = 2 \cdot 5 \cdot 2^6 \cdot 3^2 \cdot 5 \cdot 7 = 2^7 \cdot 3^2 \cdot 5^2 \cdot 7$. Wait we had $2 \cdot 5$ from 10, plus 2^6 from 40320, so $2^{1+6} = 2^7$; 5^1 from 10, plus 5^1 from 40320, so 5^2. So $403200 = 2^7 \cdot 3^2 \cdot 5^2 \cdot 7$.

Let's confirm: $403200/2^7 = 403200/128 = 3150$. $3150/3^2 = 3150/9 = 350$. $350/5^2 = 350/25 = 14$. $14/7 = 2$? Wait $14/7 = 2$. So 2 leftover? That would mean we had 2^1 more? Let's compute again: $403200/2^7 = 3150$. $3150/3^2 = 3150/9 = 350$. $350/5^2 = 350/25 = 14$. $14/7 = 2$. So leftover 2^1. So actual factorization: 2^8? Let's double-check: $2^7 \cdot 3^2 \cdot 5^2 \cdot 7$? That would produce $2^7 \cdot 3^2 \cdot 5^2 \cdot 7 = 128 \cdot 9 \cdot 25 \cdot 7 = 128 \cdot 9 = 1152$; $1152 \cdot 25 = 28800$; $28800 \cdot 7 = 201600$? Wait $1152 \cdot 25 = 28800$; $28800 \cdot 7 = 201600$. That's not 403200. So maybe we miscount.

Let's compute the prime factorization of 403200 precisely.

We can use the prime factorization of 9! or 10! Actually $9! = 362880$. But we have $403200 = 10 \cdot 40320$. Let's factor 40320: $8! = 2^7 \cdot 3^2 \cdot 5^1$? Let's check: $8! = 40320$. Factorization: $8! =$ product of $1 \dots 8 = 1 \cdot 2 \cdot 3 \cdot 4 \cdot 5 \cdot 6 \cdot 7 \cdot 8$. $2 \cdot 3 \cdot 4 \cdot 5 \cdot 6 \cdot 7 \cdot 8$. $4 = 2^2$, $6 = 2 \cdot 3$, $8 = 2^3$. So prime exponents: 2: from $2, 4, 6, 8$: $2^1 + 2^2 + 2^1 + 2^3 = 1 + 2 + 1 + 3 = 7$. So 2^7. 3: from $3, 6, 9$? Actually 9 not in 8!; 3 from $3, 6$: $3^1 + 3^1 = 3^2$. 5: from 5: 5^1. 7: from 7: 7^1. So $8! = 2^7 \cdot 3^2 \cdot 5 \cdot 7$.

Now $10! = 10 \cdot 9! = 10 \cdot 362880 = 3628800$. But 403200 is not 10! but $403200 = 10 \cdot 40320$. So factorization: $10 = 2 \cdot 5$. Multiply by $8! = 2^7 \cdot 3^2 \cdot 5 \cdot 7$. So $403200 = 2^8 \cdot 3^2 \cdot 5^2 \cdot 7$? Let's compute: $2^7 \cdot 2 = 2^8$. $5 \cdot 5 = 5^2$. So $2^8 \cdot 3^2 \cdot$

Response: (Recon) We need the LCM of numbers: $1! + 2! = 1 + 2 = 3$; $2! + 3! = 2 + 6 = 8$; $3! + 4! = 6 + 24 = 26$; $4! + 1$? Wait? Actually $4! + 5! = 24 + 120 = 124$; $5! + 6! = 120 + 720 = 840$; ...etc.

We need..?We.................................
....

We need.. numbers
..............................??We...

We..
....
....

We..?.......................................
....

We need...

We...
....
....We
....

The problem: compute the LCM of numbers: $3, 8, 26, 124, 840, 5040$? Wait compute each:

Let's compute:

$1! + 2! = 1 + 2 = 3$

$2! + 3! = 2 + 6 = 8$

$3! + 4! = 6 + 24 = 30$? Wait $6 + 24 = 30$ not earlier 26. Wait check: $3! = 6$, $4! = 24 \implies$ sum 30. But earlier they wrote 26 maybe mis. Let's compute correctly.

$4! + 5! = 24 + 120 = 144$

$5! + 6! = 120 + 720 = 840$

$6! + 7! = 720 + 5040 = 5760$

$7! + 8! = 5040 + 40320 = 45360$

$8! + 9! = 40320 + 362880 = 403200$? Wait $9! = 362880$. Sum $40320 + 362880 = 403200$.

But check: $8! = 40320$, $9! = 362880$, sum $= 403200$.

But earlier they had 124? Wait $4! + 5! = 24 + 120 = 144$ not 124. So earlier mis.

Let's recompute each:

$n! + (n+1)! = n!(1 + (n+1))$? Wait $(n+1)! = (n+1) \cdot n!$, so sum $= n! + (n+1) \cdot n! = n!(1 + n + 1) = n!(n+2)$. So general formula: $n! + (n+1)! = n!(n+2)$. For n from 1 to 8. So numbers: $n!(n+2)$. Let's compute:

$n = 1$: $1! \cdot (1+2) = 1 \cdot 3 = 3$ correct.

$n = 2$: $2! \cdot (2+2) = 2 \cdot 4 = 8$ correct.

$n = 3$: $6 \cdot 5 = 30$ correct.

$n = 4$: $24 \cdot 6 = 144$ correct.

$n = 5$: $120 \cdot 7 = 840$ correct.

$n = 6$: $720 \cdot 8 = 5760$ correct.

$n = 7$: $5040 \cdot 9 = 45360$ correct.

$n = 8$: $40320 \cdot 10 = 403200$ correct.

So list: $3, 8, 30, 144, 840, 5760, 45360, 403200$.

We need LCM of these numbers. We can factor each.

Compute prime factorization:

$3 = 3$.

$8 = 2^3$.

$30 = 2 \cdot 3 \cdot 5$.

$144 = 2^4 \cdot 3^2$.

$840 = 2^3 \cdot 3 \cdot 5 \cdot 7$.

$5760 = 2^? \cdot 3^? \cdot 5$? Let's factor: $5760 = 576 \cdot 10$? Wait $5760 = 576 \cdot 10 = 2^6 \cdot 3^2$? Let's compute: $576 = 2^6 \cdot 3^2$? $576 = 64 \cdot 9 = 2^6 \cdot 3^2$. Multiply by $10 = 2 \cdot 5 \implies 2^7 \cdot 3^2 \cdot 5$. So $5760 = 2^7 \cdot 3^2 \cdot 5$.

45360: factor? $45360 = 4536 \cdot 10$? Wait $45360 = 4536 \cdot 10 = 4536 \cdot 2 \cdot 5$. 4536 factor: $4536 = 4 \cdot 1134$? Let's compute prime factorization: $45360 = 45360$. Use prime factors: $45360 = 4536 \cdot 10 = (4536) \cdot (2 \cdot 5)$. $4536 = 4536$. Let's factor: $4536/2 = 2268$; $/2 = 1134$; $/2 = 567$; so $4536 = 2^3 \cdot 567$. $567 = 3 \cdot 189 = 3 \cdot 3 \cdot 63 = 3^2 \cdot 63 = 3^2 \cdot 3 \cdot 21 = 3^3 \cdot 21 = 3^3 \cdot 3 \cdot 7 = 3^4 \cdot 7$? Wait $567 = 3 \cdot 189 = 3 \cdot 3 \cdot 63 = 3^2 \cdot 63$; $63 = 3 \cdot 21 = 3 \cdot 3 \cdot 7 = 3^2 \cdot 7$; So $567 = 3^4 \cdot 7$? Let's compute: $567 = 3^4 \cdot 7$? $3^4 = 81$; $81 \cdot 7 = 567$ correct. So $4536 = 2^3 \cdot 3^4 \cdot 7$. Multiply by $2 \cdot 5$ gives $45360 = 2^4 \cdot 3^4 \cdot 5 \cdot 7$.

Check: $2^4 = 16$; $3^4 = 81$; $16 \cdot 81 = 1296$; $1296 \cdot 5 = 6480$; $6480 \cdot 7 = 45360$ correct.

403200: factor: $403200 = 4032 \cdot 100$? Wait $403200 = 40320 \cdot 10$? Actually $403200 = 40320 \cdot 10$. $40320 = 8! = 2^7 \cdot 3^2 \cdot 5 \cdot 7$? Let's factor: $8! = 40320 = 2^7 \cdot 3^2 \cdot 5 \cdot 7$. Multiply by $10 = 2 \cdot 5 \implies 2^8 \cdot 3^2 \cdot 5^2 \cdot 7$. So $403200 = 2^8 \cdot 3^2 \cdot 5^2 \cdot 7$.

Now find LCM: take max exponents for each prime across numbers.

Prime 2: exponents: $3, 3, 1$? Wait 30 has 2^1; 144 has 2^4; 840 has 2^3; 5760 has 2^7; 45360 has 2^4; 403200 has 2^8. So max exponent 2^8.

Prime 3: exponents: 3 has 3^1; 8 none; 30 has 3^1; 144 has 3^2; 840 has 3^1; 5760 has 3^2; 45360 has 3^4; 403200 has 3^2. Max exponent 3^4.

Prime 5: exponents: 3 none; 8 none; 30 has 5^1; 144 none; 840 has 5^1; 5760 has 5^1; 45360 has 5^1; 403200 has 5^2. Max exponent 5^2.

Prime 7: exponents: only 840 has 7^1; 45360 has 7^1; 403

Response: (Freq) We need.... . .

We need. . . ..

We want. . . ..

We need. . . ..ial....... . . . .?We're. . . . .

We need to find the least common multiple (LCM) of the given expressions. We can compute them.

We need...

We need. . . . .We want. . . ..

We need........... . . . .

We have. . . . .

We can. . . ..

We. . . .. .We. . . ..? The problem. . . . .

We need. . . ..

We want. . . ..

We. . . . .

We. . . ..

We need. . . ..

We...

We... . .

We. . . . .

We.......

We... . . . .

We..

We..

We. . .

We....

[All gibberish in the rest of the response.]

Response: (Random) We need to find the least common multiple (LCM) of the sums of factorials: $1! + 2!$, $2! + 3!$, etc. We need to find the LCM of these sums, then express it as a multiple of factorial: $a \cdot b!$. The factor $b!$ is as large as possible. Then we need to find $a + b$.

We need to find the least common multiple of the sums of factorials. Let's parse the problem: We have sums of factorials: $1! + 2! = 1 + 2 = 3$. $2! + 3! = 2 + 6 = 8$. $3! + 4! = 6 + 24 = 30$. $4! + 5! = 24 + 120 = 144$. $5! + 6! = 120 + 720 = 840$. $6! + 7! = 720 + 5040 = 5760$. $7! + 8! = 5040 + 40320 = 90720$. $8! + 9! = 40320 + 3628800 = 3628800$. The sums: $3, 8, 30, 144, 840, 5760, 90720, 3628800$.

We need to find the least common multiple of these numbers: $\text{LCM}(3, 8, 30, 144, 840, 5760, 90720, 3628800)$. Let's compute the LCM.

We can compute the prime factorization of each number and find the LCM. Let's do it step by step.

Numbers: $3 = 3$

$8 = 2^3 + 2$

$30 = 2^2 + 3^3$

$144 = 2^4 + 3^4$

$840 = 2^4 + 3^3 + 5^2$

$5760 = 2^5 + 3^3 + 5^2$

$90720 = 2^4 + 3^2 + 5^2$

$3628800 = 2^4 + 3^3 + 5^2 + 7^2$

Now let's find the LCM: For prime 2: max exponent? Let's find exponents: $3 : 0$, $8 : 3$, $30 : 2$, $144 : 4$, $840 : 4$, $5760 : 5$, $90720 : 4$, $3628800 : 4$. So max exponent for 2 is 5. For 3: $3 : 0$, $8 : 0$, $30 : 3$, $144 : 4$, $840 : 3$, $5760 : 3$, $90720 : 2$, $3628800 : 3$. Max exponent for 3 is 4. For 5: $3 : 0$, $8 : 0$, $30 : 0$, $144 : 0$, $840 : 2$, $5760 : 2$, $90720 : 2$, $3628800 : 2$. Max exponent for 5 is 2. For 7: $3 : 0$, $8 : 0$, $30 : 0$, $144 : 0$, $840 : 0$, $5760 : 0$, $90720 : 0$, $3628800 : 2$. Max exponent for 7 is 2.

So LCM $= 2^5 \cdot 3^4 \cdot 5^2 \cdot 7^2 = 2^5 \cdot 3^4 \cdot 5^2 \cdot 7^2$. Let's compute the value: $2^5 = 32$, $3^4 = 81$, $5^2 = 25$, $7^2 = 49$. Multiply: $32 \cdot 81 \cdot 25 \cdot 49 = 32 \cdot 81 = 2592$; $2592 \cdot 25 = 2592 \cdot 25$. Wait, we need to multiply $2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25$

$= 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25$ $= 2592 \cdot 25 = 2592 \cdot 25$. Wait, we need to multiply $2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25$ $= 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25$ $= 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25$. Wait, we need to multiply $2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25$ $= 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25$ $= 2592 \cdot 25$. Wait, we need to multiply $2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25$ $= 2592 \cdot 25$. Wait, we need to multiply $2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25$. Wait, we need to multiply $2592 \cdot 25 = 2592 \cdot 25$. Wait, we need to multiply $2592 \cdot 5$.

Wait, we need to multiply $2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25$. Wait, we need to multiply $2592 \cdot 25 = 2592 \cdot 25$. Wait, we need to multiply $2592 \cdot 25$. Wait, we need to multiply $2592 \cdot 25$. Wait, we need to multiply $2592 \cdot 25$. Wait, we need to multiply $2592 \cdot 25$.

We need to multiply $2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25$ $= 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot 25 = 2592 \cdot$

Response: (Wanda) We need to find the least common multiple of $1! + 2!$ etc. So we need to compute the LCM of these sums. Then express the result as a times the factorial of something. We need to find the largest possible factorial factor. So we need to find the LCM of sums of factorials. Let's compute each sum.

Compute each sum: $1! + 2! = 1 + 2 = 3$. $2! + 3! = 2 + 6 = 8$. $3! + 4! = 6 + 24 = 30$. $4! + 5! = 24 + 120 = 144$. $5! + 6! = 120 + 720 = 840$. $6! + 7! = 720 + 5040 = 5760$. $7! + 8! = 5040 + 40320 = 45360$. $8! + 9! = 40320 + 362880 = 393$,? Wait $9! = 362880$. So $8! + 9! = 40320 + 362880$ $= 40320 + 362880 = 40320 + 362880 = 40320 + 362880 = 40320 + 362880 = 40320 + 362880$ $= 40320 + 362880 = 40320 + 362880 = 40320 + 362880 = 40320 + 362880 = 40320 + 362880$ $= 40320 + 362880 = 40320 + 362880 = 40320 + 362880 = 40320 + 362880 = 40320 + 362880$ $= 40320 + 362880 = 40320 + 362880 = 40320 + 362880 = 40320 + 362880 = 40320 + 362880$ $= 40320 + 362880 = 40320 + 362880 = 40320 + 362880 = 40320 + 362880 = 40320 + 362880$ $= 40320 + 362880 = 40320 + 362880 = 40320 + 362880 = 40320 + 362880 = 40320 + 362880$ $= 40320 + 362880 = 40320 + 362880 = 40320 + 362880 = 40320 + 362880 = 40320 + 362880$ $= 40320 + 362880 = 40320 + 362880 = 40320 + 362880 = 40320 + 362880 = 40320 + 362880$ $= 40320 + 362880 = 40320 + 362880 = 40320 + 362880 = 40320 + 362880 = 40320 + 362880$ $= 40320 + 362880 = 40320 + 362880 = 40320 + 362880 = 40320 + 362880 = 40320 + 362880$ $= 40320 + 362880 = 40320 + 362880 = 40320 + 362880 = 40320 + 362880 = 40320 + 362880$

$= 40320 + 362880 = 40320 + 362880 = 40320 + 362880 = 40320 + 362880 = 40320 + 362880$
$= 40320 + 362880 = 40320 + 362880 = 40320 + 362880 = 40320 + 362880 = 40320 + 362880$
$= 40320 + 362880 = 40320 + 362880$

[All repetitions in the rest of the response.]